

캡스톤디자인프로젝트 B

17팀 CtrlS

난독 아동의 읽기 개선을 위한,
음성 인식 기반 읽기 오류 패턴 분석 및
맞춤형 동화책 생성을 활용한 학습 서비스



산학트랙

지도교수 윤명국 교수님

2183019 컴퓨터공학과 정유진

2244010 컴퓨터공학과 복지희

2276289 컴퓨터공학과 정윤아

1. 팀 정보

1.1. 과제명

난독 아동의 읽기 개선을 위한, 음성 인식 기반 읽기 오류 패턴 분석 및 맞춤형 동화책 생성을 활용한 학습 시스템

1.2. 팀 정보

17팀, CtrlS

1.3. 팀 구성원

복지희 (2244010): 리더, Full-Stack, AI(STT, 적응형 알고리즘 설계)

정유진(2183019): 팀원, Backend, AI(동화 생성, 적응형 알고리즘 설계)

정윤아(2276289): 팀원, Full-Stack, AI(이미지 생성, TTS)

2. 과제 요약

2.1 문제 정의

난독증은 지능이나 학습 의지와 무관하게 문자를 소리로 변환하는 해독 과정 자체에 신경학적 결함이 발생하는 읽기장애로, 아동의 학업 성취와 정서 발달 전반에 걸쳐 복합적인 영향을 미친다.

서울시교육청 통계에 따르면 난독증으로 지원을 받은 학생 수는 2021년 112명에서 2024년 958명으로 약 10배 가까이 급증하였다. 국내외 연구에 따르면 소아·청소년 난독증의 유병률은 약 17.5%에 달하며, 학습장애로 일상생활에 어려움을 겪는 아동의 80% 이상이 난독과 직접적인 관련이 있는 것으로 보고된다. [1]

특히 코로나19 팬데믹 이후 교육 공백과 미디어 노출 증가가 맞물리며 초등학생 난독 진단 건수가 3년 만에 9배 수준으로 급증하였고, 각 교육청이 예산 부족을 이유로 치료 횟수를 축소하거나 조기 중단하는 사례가 늘어나면서 적기 치료를 받지 못한 아동이 회복 불가 상태에 이를 수 있다는 우려가 교육 현장에서 제기되고 있다. [2] 이는 난독증이 일부 아동에게만 해당하는 개별적 문제가 아니라, 교육 시스템 전반에서 빠르게 확산되고 있는 구조적 문제임을 보여준다.

1) 학습적 문제

- 글자와 소리를 대응시키는 음운 해독 과정에서 대치·생략·삽입 오류가 반복적으로 발생함
- 읽기 능력은 국어에 국한되지 않고 수학 문제 이해, 사회·과학 교과서 해석 등 전 교과 학습의 토대가 되므로, 난독으로 인한 학습 격차가 시간이 지날수록 전 과목으로 확산됨
- 관심 주제의 동화책조차 혼자 읽어내지 못하는 경험이 반복되어 자기주도적 독서 습관 형성 자체가 어려워짐

2) 정서적 문제

- 글자가 흔들려 보이거나 단어 경계가 불분명하게 인식되는 지각적 어려움으로 읽기 상황 자체를 회피하려는 경향을 보임
- 학급에서 소리 내어 읽는 상황에서 반복적으로 실수하며 또래 앞에서 수치심을 느끼고, 점차 수업 참여를 기피하게 됨
- 실패 경험이 누적되면서 "나는 읽지 못하는 아이"라는 부정적 자기 인식이 고착되고 학습 자존감이 전반적으로 저하됨
- 장기적으로는 우울·불안 증상이 심화되어 학교 부적응 및 사회적 고립으로 이어질 위험이 존재함

3) 환경적 문제

- 코로나19 이후 대면 수업 중단과 디지털 미디어 이용 증가가 복합적으로 작용하며 읽기 발달이 지연된 아동의 수가 크게 증가함
- 특수교사 인력이 수요를 따라가지 못해 학교 현장에서 아동 개별 진단 및 맞춤 지도를 실시하기 어려운 구조적 한계가 존재함
- 언어치료센터의 높은 비용 부담으로 경제적 여건이 어려운 가정에서는 지속적인 치료 유지가 어려워 계층 간 교육 격차가 심화되고 있음

4) 정보 부족 및 편견

- 난독증을 학습 태도나 노력 부족의 문제로 오해하는 사회적 편견이 여전히 존재하여 적절한 진단과 지원이 지연되는 경우가 많음
- 보호자가 아동의 구체적인 취약 음운·어휘 유형을 파악할 수 있는 객관적 수단이 부족함
- 정량적 평가 지표 없이 감각적 관찰에만 의존한 지도가 이루어져 학습 개선 효과를 확인하기 어려움

이처럼 난독 아동이 직면한 문제는 학습·정서·환경·정보 측면에서 복합적으로 얽혀 있으며, 단순한 교재 제공이나 일회성 치료로는 근본적인 해결이 어렵다. 아동 개개인의 읽기 오류 패턴, 학습 진전도, 인지 발달 수준을 실시간으로 반영하여 맞춤형 학습 콘텐츠와 구체적인 피드백을 지속적으로 제공하는 통합 학습 시스템의 필요성이 제기된다.

2.2. 기존 서비스와의 비교

난독 아동을 대상으로 하는 기존 읽기 학습 서비스 및 유사 에듀테크 서비스와 본 서비스를 비교함으로써, 본 과제와 차별성을 제시한다.

1) 비교 대상 서비스 개요

- Grapho Game

Grapho Game은 핀란드에서 개발된 난독증 아동 대상 읽기 훈련 앱으로, 글자와 소리를 연결하는 음운 인식 훈련을 게임 형식으로 제공한다. 초등학교 1학년 때 정기적으로 사용한 난독 아동이 3학년 초에 보충 교육만 받은 아동보다 우수한 읽기 성적을 보인다는 연구 결과가 보고된 바 있어 교육적 효과가 검증된 서비스이다. 그러나 한국어를 지원하지 않으며, 사전 설정된 콘텐츠를 순서대로 제공하는 방식으로 아동 개인의 읽기 오류 데이터를 실시간으로 반영하는 적응형 학습 구조가 부재하다.

- 웅진 스마트올 / 밀크T

웅진 스마트올, 천재교육 밀크T 등은 국내 대표적인 아동 대상 학습 플랫폼으로, AI 기반 맞춤형 학습 추천 기능을 제공한다. 그러나 이들 서비스는 문제 풀이 정답률 기반의 난이도 조절에 초점을 맞추고 있으며, 난독 음성 분석을 통한 읽기 오류 패턴 추출 기능은 제공하지 않는다. 난독증과 같은 읽기 장애를 가진 아동을 위한 특화 기능이 없으며, 콘텐츠 역시 아동의 관심사나 읽기 수준에 맞춰 동적으로 생성되지 않는다.

- 리딩게이트

리딩게이트는 아동의 읽기 수준에 맞는 영어 도서를 추천하고 독해 퀴즈를 제공하는 플랫폼이다. 읽기 수준 진단 및 단계별 콘텐츠 제공이라는 점에서 일부 유사한 방향성을 가지나, 영어 전용 서비스로 한국어 난독증 아동에게는 적용이 불가능하다. 또한 음성 인식 기반의 난독 오류 분석 기능 없이 퀴즈 점수 중심으로만 학습 수준을 판단한다.

비교 항목	Grapho Game	웅진 스마트올 / 밀크T	리딩게이트	토토리 (본 과제)
한국어 지원	×	✓	×	✓
난독 음성 분석 (STT)	×	×	×	✓ (WCPM, 오류 유형 분류)
읽기 오류 패턴 정량화	×	×	×	✓
맞춤형 콘텐츠 자동 생성	×	일부 (추천 방식)	일부 (추천 방식)	✓ (LLM 기반 동화 생성)
아동 관심사 반영	×	×	×	✓
삽화 생성 (이미지 AI)	×	×	×	✓ (Stable Diffusion)
보호자/교사 리포트	×	✓ (풍부한 학습 이력)	×	✓ (읽기 특화 리포트)
적응형 난이도 자동 조절	×	✓ (검증된 커리큘럼 기반)	일부	✓ (L1~L6 자동 조정)
콘텐츠 양 및 커리큘럼 체계	✓ (다년간 누적 콘텐츠)	✓ (교과 연계 체계)	✓ (단계별 도서 다수)	△ (생성형 AI로 보완)
플랫폼 지원	✓ (iOS·Android)	✓ (iOS·Android·PC)	✓ (iOS·Android·Web)	△ (iOS 한정, 초기 버전)

2.3. 제안 내용 및 최종 산출물 (Solution)

본 과제는 난독 아동이 겪는 읽기 어려움을 기술적으로 완화하고, 아동 개개인의 읽기 능력에 부합하는 개인 맞춤형 학습 경험을 제공하는 AI 기반 모바일 학습 서비스 토토리를 개발하는 것을 제안한다.

본 서비스는 음성 인식(STT), 대규모 언어 모델(LLM), 이미지 생성 AI, 음성 합성(TTS) 기술을 결합하여 진단 → 생성 → 훈련의 순환 학습 구조를 구현한다.

1) 구체적 해결책

- 관심사 기반 맞춤형 동화 자동 생성

아동이 좋아하는 키워드를 음성으로 입력하면, ~~GPT-4o~~ **GPT-5.1** 기반 LLM이 해당 주제에 맞는 동화 텍스트를 자동 생성한다. 생성된 텍스트는 KoNLPy 형태소 분석기를 통해 어휘 난이도, 문장 길이, 품사 비율 등을 분석하여 아동의 현재 읽기 수준(L1~L6)에 맞게 자동 조정된다. 동시에 Stable Diffusion API를 활용해 동화 장면별 삽화를 생성함으로써 아동의 읽기 몰입도를 높인다.

- 난독 음성 기반 읽기 오류 패턴 분석

아동이 동화를 소리 내어 읽는 동안 STT 모듈(Whisper)이 난독 음성을 실시간으로 텍스트로 변환하고, Levenshtein Distance 알고리즘을 적용하여 원문과의 차이를 비교 분석한다. 이를 통해 대치·생략·삽입 오류 유형을 자동으로 분류하고, 타임스탬프 기반으로 분당 정확하게 읽은 단어 수(WCPM)를 정량적으로 산출한다. 도출된 오류율, 유창성 지표, 음운 취약점 데이터는 데이터베이스에 누적 저장된다.

- 적응형 난이도 자동 조절 (L1~L6)

6단계(L1~L6)로 구성된 자체 읽기 수준 체계를 기반으로 학습 난이도를 자동 조절한다. 각 단계는 대상 연령, 목표 WCPM, 최대 문장 길이, 동화 페이지 수를 기준으로 설계하였으며, 최근 3회 학습 데이터의 이동평균을 산출하여 실력 변화를 판정한 후 아래 기준에 따라 단계를 조정한다.

단계	대상 연령	목표 WCPM	최대 문장 길이	동화 페이지 수
L1	3~4세	15	4어절	18
L2	5~6세	30	5어절	18
L3	7~8세	50	7어절	21
L4	9~10세	80	13어절	21
L5	11~12세	110	15어절	24
L6	13세	130	17어절	24

- 오류 유형 기반 맞춤형 퀴즈 및 반복 학습

낭독 과정에서 수집된 취약 음운 패턴을 기반으로 LLM이 개인화된 단어 퀴즈를 자동 생성한다. 아동이 반복적으로 오류를 보이는 음절 패턴(예: 받침 'ㄱ' 생략)을 포함한 단어가 다음 동화 및 퀴즈에 적정 빈도로 삽입되어 자연스러운 반복 교정 기회를 제공한다.

- 보호자 대상 학습 리포트 제공

누적된 학습 데이터를 바탕으로 WCPM 변화 추이, 오류 유형 분포, 읽기 정확도 변화를 시각화한 주간 및 전체 리포트를 보호자에게 제공한다. 이를 통해 감각적 판단에 의존하지 않고 객관적 데이터 기반의 지도가 가능해진다.

2) 최종 산출물

본 과제의 최종 산출물은 난독 아동을 위한 iOS 모바일 애플리케이션 토토리이다. 해당 앱은 동화 생성, 낭독, 퀴즈, 학습 리포트의 네 가지 핵심 기능을 하나의 플랫폼에 통합하여 제공한다.

앱의 핵심 엔진으로는 STT(Whisper)와 Levenshtein Distance 알고리즘을 결합한 읽기 오류 분석 파이프라인, ~~GPT-4o~~ **GPT-5.1**와 KoNLPy 형태소 분석기를 활용한 적응형 동화 생성 엔진, 그리고 Stable Diffusion API 기반의 장면별 삽화 자동 생성 모듈이 구현된다.

백엔드는 Spring Boot 및 FastAPI 서버로 구성되며, AWS EC2·RDS(MySQL)·S3 클라우드 인프라 위에서 운영된다.

보호자를 위한 WCPM·정확도·오류 유형 시각화 학습 리포트 시스템 또한 최종 산출물에 포함된다.

수행 기간은 2025년 9월 1일부터 2026년 6월 4일까지 총 9개월이며, iOS 앱스토어 공식 등록 및 국내외 AI·에듀테크 관련 공모전 출품을 성과 목표로 한다.

3) 최종 목표

본 과제의 최종 목표는 수치화된 기준을 통해 다음과 같이 제시한다.

- **읽기 오류 분류 정확도:** STT 전사 결과와 원문 비교를 통한 대치·생략·삽입 오류 분류 정확도 90% 이상 달성을 목표로 한다.
- **WCPM 측정 신뢰도:** 기준 측정값 대비 오차 ± 5 WCPM 이내로 읽기 속도를 정밀하게 산출한다.
- **콘텐츠 생성 응답 시간:** 동화 텍스트는 15초 이내, 삽화는 30초, **음성은 10초** 이내 생성을 목표로 하여 아동이 끊김 없이 학습을 이어갈 수 있도록 한다.
- **학습 난이도 체계:** L1~L6 총 6단계로 구성된 적응형 난이도 체계를 완성하며, 각 단계별 목표 WCPM 및 최대 문장 길이 기준을 적용한다.
- **읽기 정확도 향상:** 서비스 사용 4주 후 아동의 읽기 정확도가 사용 전 대비 10% 이상 향상되는 것을 목표로 한다.

- **앱 출시 및 공모전 출품:** 2026년 10월 내 iOS 앱스토어 등록을 완료하고, 동년 내 AI·에듀테크 관련 공모전 1건 이상 출품을 목표로 한다.

2.4. 기대 효과 및 의의

본 프로젝트가 성공적으로 수행될 경우, 기술적 차별성을 바탕으로 난독 아동 교육 분야에서 다음과 같은 효과를 기대할 수 있다.

1) 교육적 및 치료적 측면

- **자기주도적 읽기 동기 부여:** 아동의 관심사를 반영한 맞춤형 동화를 실시간 생성함으로써, 학습을 '놀이'로 인식하게 하여 독서 흥미를 극대화한다.
- **정교한 맞춤형 학습 지원:** 단순한 읽기 반복이 아니라 음성 인식 기술로 분석된 오류 패턴(발음, 생략, 대치 등)을 즉각 학습 콘텐츠에 반영하여 정밀한 개인 맞춤형 학습을 지원한다.
- **성공 경험을 통한 자신감 회복:** 아동의 실제 실력 향상 추이를 반영한 난이도 조절을 통해 지속적인 성공 경험을 제공하고 읽기 자신감을 고취한다.

2) 사회적 및 산업적 측면

- **교육 격차 완화:** 가정 및 보육 현장에서 전문가 없이도 활용 가능한 AI 학습 도구를 제공하여 난독 지원 체계의 접근성을 높인다.
- **객관적 진단 데이터 제공:** 보호자에게 정량화된 읽기 지표(WCPM, 정확도 등)가 담긴 리포트를 제공하여 아동의 발달 상태를 객관적으로 파악하게 돕는다.
- **서비스 확장성:** 본 기술은 향후 일반 아동의 문해력 교육이나 저연령층 맞춤형 독서 서비스로도 확장 적용이 가능하다.

2.5. 주요 기능 리스트

본 서비스에서 제안하는 주요기능은 다음과 같다.

1) 관심사 기반 동화 생성 기능

- **사용자 정의 콘텐츠 생성:** 아동이 선호하는 키워드를 음성으로 입력하면 **GPT-4o** **GPT-5.1**를 통해 해당 주제에 맞는 동화 시나리오를 자동으로 구성한다.
- **시각적 몰입도 향상:** **Stable Diffusion** 기술을 활용하여 생성된 동화의 각 장면과 문맥에 어울리는 맞춤형 삽화를 실시간으로 생성함으로써 아동의 학습 참여도를 높인다.
- **텍스트 최적화 검증:** **KoNLPy** 형태소 분석기를 사용하여 생성된 동화 초안의 어휘 난이도, 품사 비율, 문장 길이를 분석하고 난독 아동에게 적합한 수준으로 자동 조정한다.

2) 음성 기반 읽기 오류 패턴 분석 기능

- **정밀 음성 인식:** **Whisper STT** 엔진을 활용하며, 자동 교정 기능을 억제하는 파라미터 조절을 통해 아동이 실제로 발음한 오류를 왜곡 없이 추출한다.
- **오류 유형 자동 분류:** 인식된 텍스트와 정답 문장을 **편집 거리(Levenshtein Distance)** 알고리즘으로 대조하여 대치, 생략, 삽입 등 난독 아동의 전형적인 오류 패턴을 분석한다.
- **유창성 정량 지표 산출:** 개별 어절의 타임스탬프 데이터를 기반으로 읽기 속도를 측정하며, 분당 정확하게 읽은 단어 수(WCPM)를 산출하여 데이터베이스에 저장한다.

3) 개인 맞춤형 적응형 난이도 조절 기능

- **6단계 표준 난이도 체계:** 인지 발달 단계에 따라 L1(3~4세)부터 L6(13세)까지 목표 WCPM와 문장 길이를 표준화하여 관리한다.
- **이동평균 기반 로직:** 단발성 세션의 결과로 난이도가 급변하는 것을 방지하기 위해 **최근 3회 학습 데이터의 이동평균**을 산출하여 실제 실력 향상 여부를 판정한다.
- **동적 레벨링:** 읽기 정확도가 90% 이상이고 목표 WCPM를 달성하면 단계를 상향하며, 정확도가 50% 미만이거나 속도가 현저히 느릴 경우 문장 길이를 축소하고 난이도를 하향 조정한다.

4) 오류 유형 기반 맞춤형 반복 학습(퀴즈) 기능

- **음운 인지 퀴즈:** 아동의 낭독 과정에서 수집된 취약 음절(예: 받침 'ㄱ' 생략 등)을 반영하여 유사한 발음이나 형태를 가진 단어들을 학습할 수 있는 환경을 제공한다.
- **다중 감각 노출:** 취약 단어를 보고, 듣고, 읽는 활동을 수행하며 동화 읽기 과정과 병행하여 자연스러운 교정을 유도한다.

5) 보호자 대상 학습 분석 리포트 기능

- **성장 추이 시각화:** 누적된 학습 데이터를 바탕으로 정확도와 읽기 속도(WCPM)의 변화 추이를 **시각적인 그래프로** 제시한다.
- **정밀 진단 근거:** 아동이 자주 틀리는 특정 음운 패턴이나 어휘 유형을 분석하여 보호자가 아동의 읽기 수준을 객관적으로 파악할 수 있는 지표를 제공한다.
- **지속 가능한 관리:** 가정과 교육 현장에서 아동의 학습 진전도를 공유함으로써 효과적인 난독 지원 체계를 마련하는 기초 자료로 활용한다.

3. 과제 설계

3.1. 요구사항 정의

1) 사용자 관점 요구사항

- **개인화 된 관심사 반영**
 - 사용자는 동화 생성 전 공룡, 우주, 동물 등 자신이 선호하는 주제 키워드를 선택할 수 있어야 한다.
 - 시스템은 선택된 키워드를 주축으로 매번 새로운 줄거리의 동화를 생성하여 학습자의 지속적인 흥미를 유도해야 한다.
- **인터랙티브 실시간 피드백**
 - 아동이 문장을 읽는 동안 시스템은 음성을 실시간으로 인식하여 읽고 있는 텍스트를 하이라이팅 등으로 표시해야 한다.
- **아동친화적 UI/UX**
 - 텍스트 위주의 메뉴 구성을 지양하고, 직관적인 아이콘과 캐릭터 중심의 인터페이스를 제공하여 문해력이 낮은 아동도 혼자 사용할 수 있어야 한다.
 - 모든 활동 안내 및 피드백은 TTS(Text-to-Speech)를 통한 음성 안내를 병행하여 인지 부담을 최소화해야 한다.

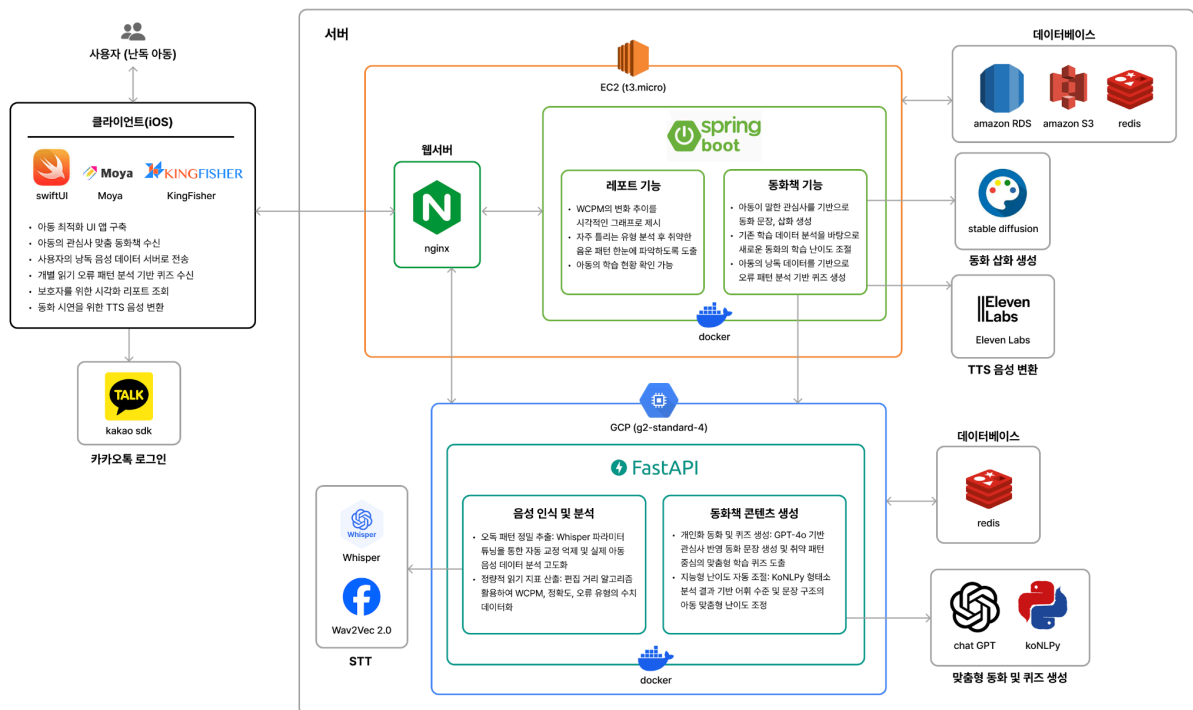
2) 시스템 관점 요구사항

- **정밀 음성 인식 및 분석:**
 - Whisper 모델 최적화: 표준어 교정 기능을 비활성화 처리하여, 난독 아동 특유의 발음 오류나 더듬거림이 필터링되지 않고 Raw Data 형태로 추출되도록 구현한다.
 - 정밀 지표 산출: 각 어절 단위의 시작/종료 타임스탬프를 추출하여, 전체 읽기 시간 대비 정확히 읽은 단어 수를 계산하는 WCPM 산출 알고리즘을 구축한다.
- **지능형 콘텐츠 제어**
 - 콘텐츠 검증: ~~GPT-4o~~ GPT-5.1로 생성된 텍스트는 KoNLPy 형태소 분석기를 거쳐야 하며, 해당 학년 수준에 맞지 않는 어려운 단어나 복잡한 문장 구조는 자동으로 순화 및 조정되어야 한다.
 - 적응형 로직(Adaptive Algorithm): 단발성 컨디션 난조로 인한 데이터 왜곡을 방지하기 위해 최근 3회의 학습 성취도를 이동 평균(Moving Average)으로 계산하여 점진적인 난이도 조절을 수행한다.

- 데이터 인프라

- 데이터베이스 설계: 아동의 프로필, 누적 오류 패턴, 학습 이력 등 영구 저장이 필요한 데이터는 MySQL에 관리하며, 실시간 세션 데이터 및 빠른 피드백이 필요한 난이도 파라미터는 Redis를 활용해 캐싱 처리한다.
 - 클라우드 인프라: AWS(Amazon Web Services) 환경을 기반으로 서버를 구축하여, 생성형 AI 모델 연동 및 대용량 음성 데이터 처리에 안정적인 확장성과 보안성을 확보한다.

3.2 전체시스템 구성



sw 구조 변경

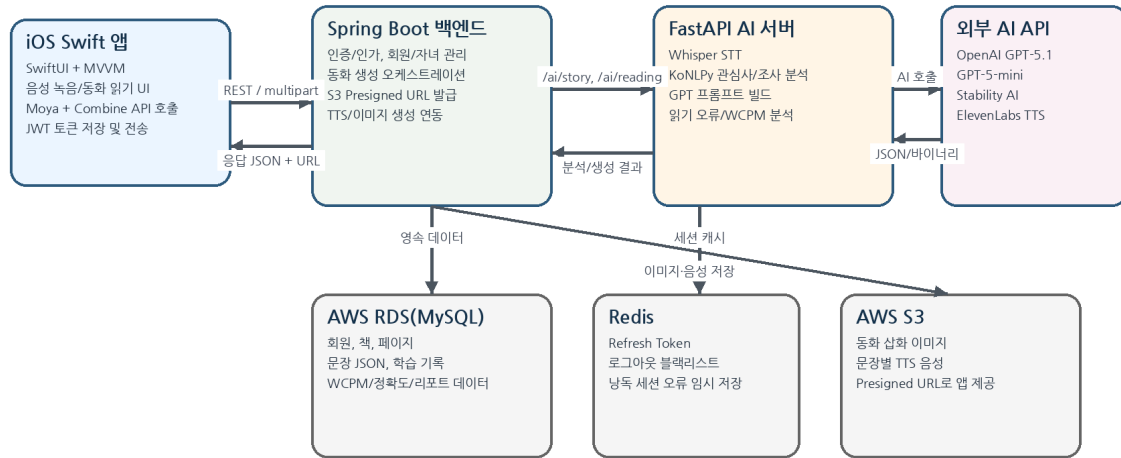
위 그림은 토토리 서비스의 전체 SW 구조를 나타낸다. iOS 클라이언트는 사용자의 음성 입력, 동화 조회, 난독, 퀴즈 풀이 요청을 Spring Boot 백엔드로 전송한다. Spring Boot 서버는 사용자 인증, 회원/동화/학습 기록 관리, S3 파일 저장 및 Presigned URL 발급을 담당하며, AI 처리가 필요한 요청은 FastAPI 서버로 전달한다. FastAPI 서버는 Whisper STT, KoNLpy 기반 전처리, GPT 기반 동화·퀴즈 생성, 오류 분석 로직을 수행한다. 생성된 동화 텍스트와 분석 결과는 Spring Boot를 통해 MySQL에 저장되고, 이미지와 음성 파일은 S3에 저장된 뒤 클라이언트로 반환된다.

- LLM (GPT-4o, GPT-5.1 & GPT-5-mini)

- 사용자 키워드 기반 동화 스크립트 생성: 아동이 입력한 관심사 키워드를 기반으로 동화 시나리오를 구성한다.
- KoNLPy 기반 형태소 분석 및 난이도 조절: LLM이 생성한 동화 초안을 KoNLPy 형태소 분석기에 전달하여 분석기는 어휘 난이도, 품사 비율, 문장 길이(평균 어절 수), 접속어/종속절 포함 여부를 계산한다. 분석 결과를 기반으로 난독 아동에게 어려운 부분을 탐지하고, 문장의 난이도를 조정한다.
- 문맥 기반 퀴즈 및 피드백 자동 생성: 아동이 자주 틀리는 단어의 유형을 반복적으로 학습하거나, 생성된 동화 내용을 바탕으로 독해력을 확인하는 퀴즈를 생성한다.
- **STT (Whisper / wav2vec 2.0)**
 - 모델 비교 및 최적화: Whisper의 자동 교정 기능을 억제하기 위해 텍스트 정규화를 비활성화하도록 파라미터를 조절한다. Whisper와 wav2vec 2.0 모델의 아동이 실제로 발음한 오류가 포함된 텍스트를 그대로 추출하는 성능을 비교 분석하여 모델을 선정하고 고도화한다.
 - 정량적 오류 지표 산출: 전사 결과와 정답 문장을 편집 거리(Levenshtein Distance) 알고리즘으로 대조하여 WCPM, 정확도, 오류 유형(대치, 생략, 삽입)을 정량화한다.
- **이미지 생성 (Stable Diffusion)**
 - LLM이 생성한 동화의 각 장면별 상황을 프롬프트로 변환하여, 이야기 전개에 맞는 삽화를 생성한다.
- **TTS (ElevenLabs)**
 - 동화 텍스트를 실시간 구연동화를 구현한다.
 - 서비스에 등장하는 텍스트를 음성으로 변환한다.
- **클라우드 서버**
 - AWS EC2 (t3.micro): 메인 백엔드 서버(Spring Boot) 구동
 - ~~AWS EC2 (g4dn.xlarge): GPU 기반 AI 서버(FastAPI, Whisper(STT), Adaptive Learning Engine) 구동~~
 - GCP (L4): GPU 기반 AI 서버(FastAPI, Whisper(STT), Adaptive Learning Engine) 구동
 - AWS RDS (MySQL): 사용자 데이터 저장을 위한 DB 서비스
 - AWS S3: 동화 삽화 이미지 저장

3.3 주요 엔진 및 기능 설계

본 서비스의 핵심 기능을 구현하기 위해 도입한 주요 엔진과 백엔드 인증 모듈을 SW 모듈 단위로 기술한다. 각 항목은 단순 사용 기술 소개가 아니라 실제 구현에서 어떤 모듈을 만들었고, 해당 모듈이 어떤 입력을 받아 어떤 출력을 생성하는지를 중심으로 정리하였다.



3.3.1 프론트엔드

Totori 앱의 iOS 프론트엔드는 SwiftUI 기반의 MVVM(Model-View-ViewModel) 아키텍처로 구현되었다. 네트워크 통신은 Moya + Combine을 사용하며, 이미지 캐싱은 Kingfisher 라이브러리를 활용한다.

(1) MVVM 패턴 및 기술 스택

iOS 프론트엔드는 SwiftUI의 선언형 UI 프레임워크와 MVVM 패턴을 결합하여 구현하였다.

각 화면은 View(UI 선언)와 ViewModel(@ObservableObject, 비즈니스 로직)로 분리되며, ViewModel은 @Published 프로퍼티를 통해 View와 반응형으로 바인딩된다.

계층	기술/라이브러리	역할
UI 프레임워크	SwiftUI	선언형 UI 구성, @StateObject/@Published 기반 반응형 바인딩
네트워크	Moya + CombineMoya	REST API 추상화(TotoriAPI enum) 및 Combine Publisher 기반 비동기 통신
인증	Alamofire + TokenInterceptor	JWT Bearer 토큰 자동 첨부, 만료 시 자동 갱신(reissue) 인터셉터
이미지 로딩	Kingfisher (KfImage)	S3 Presigned URL 이미지 비동기 로딩, 메모리/디스크 캐싱, fade 애니메이션
토큰 저장	KeychainManager	accessToken, refreshToken을 iOS Keychain에 안전하게 저장
음성 녹음	AVFoundation (AVAudioRecorder)	아동 음성 녹음 (M4A, AAC 코덱, 12000Hz, 모노)
네비게이션	NavigationStack + NavigationState	@Published rootId 변경으로 루트 복귀 지원

위 표에서 핵심 설계 포인트는 Moya의 TotoriAPI enum을 통한 **API 추상화**이다.

모든 서버 통신 엔드포인트(login, signUp, generateBook, makeBook, mainStatus, bookList 등)를 하나의 enum으로 정의하고, BaseService<TotoriAPI>가 공통 요청/응답 처리를 담당한다. 이를 통해 네트워크 레이어의 코드 중복을 제거하고 타입 안전성을 확보하였다.

(2) 프론트엔드 모듈 구성

아래 표는 iOS 프론트엔드의 주요 화면 모듈과 각 모듈의 View-ViewModel 매핑이다.

View의 경우 MVVM 아키텍처를 사용했다.

화면	View	ViewModel	핵심 기능
메인 홈	MainView	MainViewModel	최근 동화, 동화 목록(페이지네이션), 도토리/벤티 표시
동화 생성	MakeStoryBookView	MakeStoryBookViewModel	4단계 생성 플로우, 음성 녹음, 서버 API 호출
동화 정보	BookInfoView	BookInfoViewModel	표지 이미지, 진행률, 도토리 보상 표시
동화 읽기	ReadStoryBookView	ReadStoryBookViewModel	장면별 이미지+텍스트 렌더링, 마이크 녹음, TTS 재생
퀴즈	WordLearningView / WordQuizRepeatView	WordViewModel	음운 퀴즈, 조사 퀴즈 UI 및 학습 반복
레포트	WeeklyReportView/ TotalReportView	WeeklyReportViewModel/ TotalReportViewModel	학습 통계 및 학습 현황 레포트

Totori 앱은 3-Tier 아키텍처(iOS Swift 클라이언트 → Spring Boot 백엔드 → FastAPI AI 서버)로 구성되며, 핵심 AI 엔진은 총 3개이다. 동화 텍스트를 생성하는 OpenAI GPT-5.1 엔진, 음성을 합성하는 ElevenLabs TTS 엔진, 삽화를 생성하는 Stability AI Stable Diffusion 3.5 flash 엔진이 그것이다. 특히, GPT 호출과 프롬프트 생성 로직은 FastAPI(Python) 서버에서, TTS 및 이미지 생성 호출은 Spring Boot(Java) 서버에서 담당하는 2단 백엔드 구조를 채택하였다.

3.3.2. ElevenLabs TTS(Text-to-Speech) 엔진 설계

2-1) TTS 엔진 선택 배경

본 프로젝트의 핵심 타겟 사용자는 난독증 아동이다. 글을 유창하게 읽지 못하는 아동에게 서비스에서 등장하는 텍스트를 자연스러운 음성으로 들려주는 것이 필요하며, 동화 낭독시에도 정확한 발음을 안내하기 위해 음성으로 들려주는 것이 필수적이라고 생각했다.

이를 위해 감정이 풍부하고 한국어 억양이 자연스러운 TTS가 필요하였다. Google TTS(gTTS)는 억양이 기계적이고, Naver CLOVA TTS는 감정 표현이 제한적이라는 한계가 있었다.

ElevenLabs는 딥러닝 기반의 다국어 모델(eleven_multilingual_v2)을 통해 한국어를 포함한 자연스러운 음성 합성을 제공하며, stability와 similarity_boost 등의 파라미터로 목소리 특성을 세밀하게 조절할 수 있어 최종 선정하였다.

2-2) ElevenLabs API 연동 구조

ElevenLabs TTS 엔진은 Spring Boot 백엔드에서 두 개의 모듈로 구현된다: ElevenLabsConfig(WebClient 설정)와 ElevenLabsClient(API 호출 로직)이다.

API 엔드포인트: POST <https://api.elevenlabs.io/v1/text-to-speech/{voiceId}>

ElevenLabsConfig는 WebClient 빈을 구성하는 설정 클래스로, API 키 인증 헤더와 타임아웃을 관리한다. 아래 표는 해당 설정 파라미터이다.

설정 항목	값(application.yml)	설명
base-url	https://api.elevenlabs.io/v1	ElevenLabs REST API 기본 URL
api-key	환경 변수로 주입	xi-api-key 헤더에 자동 설정됨
timeout-seconds	60	연결 타임아웃 10초, 응답 대기 최대 60초
maxInMemorySize	10MB	음성 바이너리 응답을 메모리에 버퍼링하기 위한 크기

ElevenLabsClient는 실제 API 호출을 담당하는 컴포넌트로, 아래는 호출 시 전달하는 요청 본문 파라미터이다.

파라미터	설정 값	설명
text	동화 문장 텍스트	SentenceData에서 추출한 개별 문장
model_id	eleven_multilingual_v2	다국어(한국어 포함) 지원 음성 모델
voice_settings.stability	0.5	낮을수록 감정 표현이 풍부 (0.0~1.0)
voice_settings.similarity_boost	0.75	원본 음성 모델과의 유사도 (높을수록 일관적)
voice_settings.style	0.5	감정의 과장을 표현, 낮을수록 자연스럽고 과장이 없음
voice_setting.speed	0.80	1이 정상 속도, 낮을수록 느림

stability 값을 0.5로 설정하여 동화 낭독에 적합한 감정 풍부한 발화를 유도하면서도, similarity_boost를 0.75로 유지하여 음성의 일관성을 확보하였다. 또한 voice_setting.speed를 0.8로 낮게 설정하여 아동이 발음에 주의하며 들을 수 있도록 하였다.

2-3) TTS 처리 플로우 및 병렬 처리

TTS 생성은 TtsService에서 관리하며, CompletableFuture 기반의 병렬 처리를 이미 구현하여 전체 생성 시간을 최소화한다. 처리 플로우는 다음과 같다.

1. Book 엔티티 내 모든 BookPage를 순회하며, 각 페이지의 SentenceData 목록을 추출한다.
2. 각 문장에 대해 CompletableFuture.runAsync()로 비동기 태스크를 생성한다.
 - ExecutorService.newFixedThreadPool(parallelism)으로 고정 크기 스레드 풀 사용 (기본 3)
 - 이미 음성이 생성된 문장(sentence.hasAudio())은 건너뛰다
3. 각 태스크에서 ElevenLabsClient.synthesize(text) 호출 → MP3 byte[] 수신
4. S3AudioStorageService.uploadAudio()로 S3에 업로드 (파일명: {bookId}_{pageOrder}_{sentenceIdx}.mp3)
5. SentenceData.updateAudio()로 음성 파일명을 엔티티에 기록
6. CompletableFuture.allOf(...).join()으로 모든 문장 TTS 완료 대기
7. 개별 문장 TTS 실패 시 해당 문장만 스킵하고 나머지는 정상 처리 (장애 격리)

2-4) ElevenLabsClient 구현 코드

아래는 실제 프로젝트에서 사용하는 ElevenLabsClient.java의 핵심 코드이다.

```
@Component
@RequiredArgsConstructor
public class ElevenLabsClient {
    private final @Qualifier("elevenLabsWebClient") WebClient elevenLabsWebClient;

    @Value("${elevenlabs.default-voice-id}")
    private String defaultVoiceId;

    @Value("${elevenlabs.default-model-id}")
    private String defaultModelId;

    public byte[] synthesize(String text) {
        Map<String, Object> body = Map.of(
            "text", text,
            "model_id", defaultModelId,
            "voice_settings", Map.of(
                "stability", 0.5,
                "similarity_boost", 0.75
            )
        );

        byte[] audio = elevenLabsWebClient.post()
            .uri("/text-to-speech/{voiceId}", defaultVoiceId)
            .header("Accept", "audio/mpeg")
            .contentType(MediaType.APPLICATION_JSON)
            .bodyValue(body)
            .retrieve()
            .onStatus(HttpStatus::isError, response -> ...)
            .bodyToMono(byte[].class)
            .block();
        return audio;
    }
}
```

2-5) TtsService 병렬 처리 구현 코드

elevenLabs의 무료 계정의 경우 한번에 요청할 수 있는 요청의 개수가 2개로 제한되어 있다. 따라서 많은 병렬 처리를 요청하면 audioUrl이 null이 되는 이슈가 발생할 수 있기 때문에 parallelism은 제한인 2개보다 낮은 1개로 안전하게 처리하였다.

또한 재시도 로직을 도입하여 audioUrl이 생성되지 않았을 경우 이를 보고 tts를 재시도 한다.

```
@Service
public class TtsService {
    private final ElevenLabsClient elevenLabsClient;
    private final S3AudioStorageService s3AudioStorageService;

    @Value("${tts.parallelism:3}")
    private int parallelism;

    public void generateAllAudios(Book book) {
        ExecutorService executor = Executors.newFixedThreadPool(parallelism);
        List<CompletableFuture<Void>> futures = new ArrayList<>();
        for (BookPage page : book.getPages()) {
            for (int i = 0; i < page.getSentences().size(); i++) {
                final SentenceData sentence = page.getSentences().get(i);
                if (sentence.hasAudio()) continue; // 이미 생성된 건 스킵
                CompletableFuture<Void> future = CompletableFuture.runAsync(() ->
                {
                    byte[] audio =
                    elevenLabsClient.synthesize(sentence.getText());
                    String fileName = bookId + "_" + pageOrder + "_" + idx +
                    ".mp3";
                    s3AudioStorageService.uploadAudio(audio, fileName);
                    sentence.updateAudio(fileName, null);
                }, executor);
                futures.add(future);
            }
        }
        CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join();
        executor.shutdown();
    }
}
```

3.3.3. Stability AI 기반 이미지 생성 엔진 설계

3-1) 이미지 생성 엔진 선택 배경

동화의 몰입감을 위해 각 장면에 맞는 삽화를 자동 생성하는 기능이 필요하였다.

로컬 Stable Diffusion WebUI(AUTOMATIC1111)는 GPU 서버 구축 비용이 높고 운영이 복잡하므로, 클라우드 기반의 Stability AI REST API를 채택하였다. 특히 stable diffusion 3.5-flash 모델은 빠른 생성 속도와 안정적인 품질을 동시에 제공하며, 동화 삽화에 적합한 일러스트레이션 스타일 생성이 가능하다. 또한 이미지 한장당 비용이 저렴하여 많은 이미지가 필요한 동화 생성 서비스에 맞는 ai라고 생각하였다.

3-2) 이미지 생성 API 연동 구조

API 엔드포인트: POST <https://api.stability.ai/v2beta/stable-image/generate/sd3>

StableDiffusionService에서 API를 호출하며, 아래는 요청 파라미터이다.

파라미터	설정값	설명
prompt	"cute children's book illustration, watercolor style, " + GPT 생성 프롬프트	GPT가 생성한 장면 프롬프트 앞에 아동용 스타일 접두어를 자동 추가
model	sd3.5-flash	Stability AI SD 3.5의 경량 모델 (빠른 생성 속도)
aspect-ratio	3:7	세로형 비율 — 동화책 페이지 형태에 최적화
output_format	png	고품질 무손실 이미지 포맷
seed	책 단위 공통 seed (랜덤 정수)	하나의 동화 내 모든 삽화에 동일 seed를 부여하여 이미지 간 화풍 일관성 유지

위 표에서 가장 중요한 설계 포인트는 두 가지이다.

1) prompt에 "cute children's book illustration, watercolor style, "이라는 스타일 접두어를 자동 추가하여 아동용 수채화 일러스트 화풍을 보장한다.

2) 같은 책의 모든 이미지에 동일한 seed 값을 부여하여(BookService에서 ThreadLocalRandom.current().nextInt(10000000)으로 생성), 페이지 간 화풍 및 캐릭터 외형의 일관성을 유지한다. 다만 seed 만으로는 완전히 일관적인 캐릭터 생성이 불가능하기 때문에 prompt에 추가적으로 캐릭터의 외형을 추가하는 문구를 덧붙였다.

추가적으로 실행될 기기(아이폰)의 화면에 맞게 생성될 이미지의 비율을 가로 : 세로 = 3 : 7로 설정하였다.

3-3) 이미지 프롬프트 생성 메커니즘

Stability AI API에 전달하는 이미지 프롬프트는 GPT가 동화 텍스트와 함께 생성한다. FastAPI의 prompt_builder.py에서 GPT에게 각 scene마다 image_prompt(영문)를 출력하도록 지시하며, 특히 캐릭터 외형 일관성 규칙을 프롬프트에 명시하였다.

GPT 프롬프트의 [제약 조건]에 다음 규칙이 포함된다:

- scene이 바뀌어도 주인공의 외형, 의상은 바뀌면 안 됨 (배경, 행동, 표정, 구도만 변경 허용)
- 모든 image_prompt에 캐릭터의 나이대, 성별, 머리색/스타일, 옷차림, 소지품, 화풍을 공통적으로 포함

GPT 응답의 JSON 구조는 다음과 같다:

```
{
  "title": "동화 제목",
  "cover_image_prompt": "표지 삽화를 위한 영어 프롬프트",
  "pages": [
    {
      "page_order": 1,
      "image_prompt": "A 5-year-old girl with short brown hair,
        wearing a yellow dress, holding a red umbrella,
        watercolor style, walking through a sunny forest...",
      "sentences": ["문장1", "문장2", "문장3"]
    },
    ...
  ]
}
```

이 JSON에서 cover_image_prompt는 표지 이미지 생성에, 각 page의 image_prompt는 장면별 삽화 생성에 사용된다.

3-4) 비동기 이미지 생성 (PageImageAsyncService)

이미지 생성은 페이지 수만큼 반복 호출이 필요하므로, 이를 동기처리할 경우 동화 생성에 너무 많은 시간이 소요된다. 따라서 @Async 어노테이션 기반의 비동기 처리를 구현하였다.

PageImageAsyncService는 StableDiffusionService.generateImage()로 이미지를 생성한 뒤, ImageStorageService.uploadImage()를 통해 S3에 업로드하고, 최종 이미지 URL을 CompletableFuture로 반환한다.

```

@Service
@RequiredArgsConstructor
public class PageImageAsyncService {
    private final StableDiffusionService stableDiffusionService;
    private final ImageStorageService imageStorageService;

    @Async("imageExecutor")
    public CompletableFuture<String> generateAndUpload(
        String prompt, Long bookSeed, String fileName) {
        byte[] imageBytes = stableDiffusionService.generateImage(prompt,
bookSeed);
        String imageUrl = imageStorageService.uploadImage(imageBytes, fileName);
        return CompletableFuture.completedFuture(imageUrl);
    }
}

```

BookService에서는 표지 이미지와 모든 페이지 이미지의 CompletableFuture를 수집한 뒤, CompletableFuture.allOf(...).join()으로 전체 이미지 생성이 완료되기를 대기한다. 이를 통해 이미지 생성이 병렬로 수행되어 전체 소요 시간이 크게 단축된다.

3-5) StableDiffusionService 구현 코드

위에서 설명한 파라미터 값들을 배치하였다.

```

@Service
public class StableDiffusionService {
    private final RestTemplate restTemplate;

    @Value("${stability.api-key}")
    private String apiKey;

    public byte[] generateImage(String prompt, long seed) {
        String url = "https://api.stability.ai/v2beta/stable-image/generate/sd3";

        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.MULTIPART_FORM_DATA);
        headers.setBearerAuth(apiKey);
        headers.set("Accept", "application/json");

        String enhancedPrompt = "cute children's book illustration,
watercolor style, " + prompt;

        MultiValueMap<String, Object> body = new LinkedMultiValueMap<>();
        body.add("prompt", enhancedPrompt);
        body.add("model", "sd3.5-flash");
        body.add("aspect-ratio", "4:6");
        body.add("output_format", "png");
        body.add("seed", seed);

        ResponseEntity<Map> response =
            restTemplate.postForEntity(url, body, Map.class);
        String base64Image = (String) response.getBody().get("image");
    }
}

```

```

        return Base64.getDecoder().decode(base64Image);
    }
}

```

3.3.4 STT (Whisper)

난독 아동의 낭독 음성을 분석하기 위해 오픈소스 Whisper 모델을 도입하였다. 기존 상용 STT API(Google, Clova 등)나 Whisper의 기본 설정은 문맥에 맞게 잘못된 발음을 자동 교정하는 특성이 있어, 아동의 실제 발음 오류를 포착해야 하는 본 프로젝트의 목적에 부합하지 않는다.

이러한 세밀한 파라미터 제어를 위해 외부 API 서비스에 의존하지 않고, 직접 구축한 AI 전용 서버(FastAPI) 환경에 Python 코드를 배포하여 모델을 직접 메모리에 로드하고 구동하는 방식을 채택하였다. 아래와 같이 PyTorch를 활용해 서버의 하드웨어 가속기(GPU/CPU)를 자동 감지하고 Whisper 모델을 로드함으로써, whisper_loader.py 모듈에서 STT 파라미터를 직접 통제하고 목적에 맞는 3가지 프리셋(raw, balanced, clean)을 자체적으로 설계할 수 있었다.

```

import os
import whisper
import torch

# 앱 시작 시 환경변수를 통해 지정된 whisper 모델(예: medium)을 1번만 로드
MODEL_NAME = os.getenv("WHISPER_MODEL", "medium")

# 서버 환경에 맞춰 CUDA(NVIDIA GPU), MPS(Mac GPU), CPU 순서대로 가용 디바이스 할당
if torch.cuda.is_available():
    device = "cuda"
elif torch.backends.mps.is_available():
    device = "mps"
else:
    device = "cpu"

# 외부 API 호출 없이 로컬 서버 메모리에 모델을 직접 로드하여 추론 환경 구축
_model = whisper.load_model(MODEL_NAME, device=device)

```

4-1) 모듈 구성 및 역할

구현 모듈	주요 역할	상세
stt_router.py	API 엔드포인트	Nginx로부터 /ai/stt/transcribe 요청을 받아 임시 파일로 오디오를 저장하고 전사를 트리거한다.
whisper_loader.py	모델 로드 & 추론	하드웨어 가속기를 감지하여 모델을 로드하고, 목적에 맞는 3가지 프리셋(raw, balanced, clean) 파라미터를 제공한다.

4-2) 파라미터 조정 방법

Raw 프리셋: 아동의 발음 흔들림과 오독을 있는 그대로 추출하기 위해 언어 모델의 보정 개입을 최소화한 설계이다.

- `temperature = 0.7`: 값을 높여 단일 정답이 아닌 다양한 발음 가능성을 탐색하도록 하여 발음의 흔들림을 보존한다.
- `beam_size = 1`: 보정 탐색을 최소화하여 Whisper가 문맥을 유추해 정답을 강제로 맞추는 현상을 방지한다.
- `condition_on_previous_text = False`: 이전 세그먼트의 텍스트가 현재 음성 인식에 미치는 영향을 차단하여, 이전 문맥 때문에 현재의 발음 오류가 교정되는 것을 막는다.
- `word_timestamps = True`: 단어 단위의 정확한 시간(start, end)과 발음 확률을 추출하여 추후 UI에서의 하이라이팅 및 정밀 분석에 사용한다.

3.3.5 GPT(LLM 기반 콘텐츠 생성 엔진)

LLM 기반 콘텐츠 생성 모듈은 OpenAI API(GPT-5.1, GPT-5-mini)를 활용하여 동화 본문 생성, 퀴즈 생성을 담당한다. 두 기능 모두 `system prompt`와 `user prompt`를 분리하고, `response_format`을 JSON 객체로 강제하여 FastAPI 서버가 결과를 안정적으로 파싱할 수 있도록 설계하였다.

5-1) 모듈 구성 및 역할

구현 모듈	주요 역할	상세
LLMStoryGeneratorService	동화 생성	GPT-5.1을 호출하여 동화 제목, 표지 이미지 프롬프트, 페이지별 문장과 장면 프롬프트를 생성한다.
QuizGeneratorService	퀴즈 생성	GPT-5-mini를 호출하여 음운 오류 단어 퀴즈와 조사 오류 문장 퀴즈를 생성한다.
PromptBuilder	프롬프트 구성	ConstraintResponse를 system/user prompt로 변환한다. 문장 길이, 페이지 수, 취약 음운, 이미지 프롬프트 규칙을 동적으로 삽입한다.
ConstraintGeneratorService	레벨별 제약 설정	아동 레벨(L1~L6), 관심사, 최근 WCPM, 취약 음운을 결합하여 동화 생성 제약 조건을 산출한다.

위 표는 GPT 기능을 하나의 거대한 API 호출로 처리하지 않고, 제약 생성·프롬프트 구성·LLM 호출·응답 검증 모듈로 분리한 구조를 보여준다. 중간 표현인 ConstraintResponse를 둔 이유는 모델을 교체하더라도 레벨별 난이도 정책과 프롬프트 조립 로직을 재사용하기 위함이다.

5-2) 동화 생성 알고리즘

동화 생성은 ProfileRequest(level, interests, recent_wcpm, weak_phonemes)를 입력으로 받아 ConstraintResponse를 생성한 뒤, 이를 GPT용 프롬프트로 변환하는 단방향 파이프라인으로 동작한다. 핵심 처리 흐름은 다음과 같다.

1. 아동 프로필 수신: 클라이언트가 아동의 현재 레벨, 관심사 키워드, 최근 WCPM, 취약 음운 목록을 FastAPI 서버로 전달한다.
2. 제약 조건 산출: ConstraintGeneratorService가 LEVEL_CONSTRAINTS를 참조하여 max_sentence_len, vocab_level, total_pages, scene_count, focus_phonemes 등을 결정한다.
3. 프롬프트 생성: PromptBuilder가 system prompt에는 역할·문체·출력 형식을 고정하고, user prompt에는 레벨별 제약과 관심사 키워드를 삽입한다.
4. LLM 호출: LLMStoryGeneratorService가 GPT-5.1를 호출하여 JSON 형식의 동화 본문을 생성한다.
5. 스키마 검증: StoryResponse.model_validate_json()으로 title, cover_image_prompt, pages[] 구조를 검증한다.
6. 예외 처리: JSON 형식 또는 필수 필드가 맞지 않으면 ValidationError를 발생시켜 재시도 가능 상태로 처리한다.
- 7.

```
response = await self.client.chat.completions.create(
    model=self.model,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ],
    temperature=0.7,
    response_format={"type": "json_object"},
)

parsed_story = StoryResponse.model_validate_json(
    response.choices[0].message.content
)
```

위 구현에서 response_format을 지정한 이유는 LLM이 일반 설명문이나 마크다운을 섞어 반환할 경우 후속 이미지 생성, TTS, 페이지 저장 모듈이 모두 실패할 수 있기 때문이다. 따라서 생성 단계에서부터 JSON-only 원칙을 적용하고, 서버 측에서는 Pydantic 스키마로 2차 검증을 수행한다.

5-3) 퀴즈 생성 알고리즘

퀴즈 생성은 낭독 분석 결과에 따라 두 가지 유형으로 분기된다. L1~L3 단계에서는 자모 단위 읽기 오류를 반복 연습하기 위한 음운 단어 퀴즈를 생성하고, L4~L6 단계에서는 한국어 문장 읽기에서 빈번하게 발생하는 조사 누락·대치·첨가를 교정하기 위한 조사 문장 퀴즈를 생성한다.

퀴즈 유형	입력 데이터	처리 방법	출력
음운 단어 퀴즈	target_word, error_pattern	오류 패턴(예: ㄱ 탈락, ㅁ 첨가, ㄴ→ㄹ 대치)을 GPT-5-mini에 전달하고, 해당 자모가 포함된 단어	QuizResponse.words[4]

		4개를 생성한다. 첫 단어는 실제로 틀린 단어로 고정한다.	
조사 문장 퀴즈	JosaEvent(kind, stem, target_josa, stt_josa)	DELETION/SUBSTITUTION/INSERTION에 따라 오류 설명과 연습 조사 값을 동적으로 구성하고, 두 어절 내외의 짧은 문장을 4개 생성한다.	QuizResponse.sentences[4]
중복 제거 후처리	LLM 응답 리스트	_dedupe()에서 첫 항목을 보존하고 중복을 제거한 뒤 최대 4개로 절단한다.	중복 없는 4지선다 후보

위 표는 퀴즈 생성이 단순히 GPT에게 문제를 만들어 달라고 요청하는 방식이 아니라, 오류 분석 모듈이 산출한 대표 오류를 입력으로 받아 학습 목표가 명확한 문항을 생성하는 구조임을 보여준다. 특히 첫 문항을 실제 오답 단어로 고정하여 아동이 방금 틀린 발음을 즉시 다시 연습할 수 있게 하였다.

5-4) 기능과 GPT 모듈의 연관성

서비스 기능	관련 GPT 모듈	구현상 역할
관심사 기반 동화 생성	ConstraintGeneratorService + PromptBuilder + LLMStoryGeneratorService	아동의 관심사와 레벨 제약을 프롬프트로 변환하여, 페이지 단위 동화 JSON을 생성한다.
오류 유형 기반 퀴즈 생성	ReadingService + QuizGeneratorService	Redis에 적재된 음운/조사 오류 통계를 조회하고, 가장 빈도가 높은 오류를 기준으로 맞춤형 퀴즈를 생성한다.

GPT 모듈은 주요 기능 리스트 중 관심사 기반 동화 생성과 오류 유형 기반 맞춤형 반복 학습을 직접 구현한다. 동화 생성은 학습 시작 전에 개인화 콘텐츠를 만드는 기능이고, 퀴즈 생성은 낭독 이후 분석된 오류를 다시 학습 콘텐츠로 되돌리는 기능이다. 따라서 GPT 모듈은 토토리의 생성 → 읽기 → 분석 → 재생성의 순환 구조에서 생성과 재학습을 담당하는 핵심 엔진이다.

3.3.6 KoNLPy (한국어 형태소 분석 모듈)

KoNLPy 모듈은 GPT 호출 전에 입력을 정제하고, 낭독 분석 이후 한국어 조사 단위 오류를 세밀하게 추출하기 위해 사용된다. 본 시스템에서는 목적에 따라 두 분석기를 분리하였다. 관심사 키워드 정제에는 구어체 입력에 비교적 관용적인 Okt를 사용하고, 조사 오류 감지에는 형태소 품사 태깅이 정교한 Mecab을 사용한다.

6-1) Okt 기반 관심사 키워드 정제

아동이 “음... 저는 공룡이랑 우주가 좋아요”처럼 자연어로 관심사를 말하면, STT 결과에는 감탄사, 조사, 어미, 불필요한 표현이 함께 포함된다. InterestRefinerService는 이 문장을 GPT 동화 생성에 바로 전달하지 않고, Okt.nouns()로 명사를 추출한 뒤 불용어와 유해어를 제거하여 안전한 주제 키워드만 남긴다.

1. Okt.nouns(text)로 명사 후보를 추출한다.
2. FILLER_WORDS에 포함된 “음”, “저”, “그냥” 등 의미 없는 발화 요소를 제거한다.
3. BANNED_KEYWORDS에 포함된 폭력적·부적절한 주제어를 차단한다.
4. THEME_HINTS 사전을 통해 유사 키워드를 서비스 내부 주제 범주로 정규화한다.
5. set 또는 순서 보존 중복 제거 로직을 적용하여 List[str] 형태로 반환한다.

```
nouns = self.okt.nouns(text)
refined_keywords = []

for noun in nouns:
    if noun in FILLER_WORDS:
        continue
    if not self._is_safe_word(noun):
        continue
    refined_keywords.append(self._normalize_theme(noun))
```

이 모듈을 별도로 둔 이유는 관심사 입력 단계에서 LLM을 사용하면 비용과 응답 지연이 증가하고, 아동 발화에 포함된 부적절한 키워드가 그대로 동화 생성으로 이어질 위험이 있기 때문이다. 따라서 관심사 정제는 LLM이 아닌 규칙 기반 전처리로 구현하였다.

6-2) Mecab 기반 조사 오류 감지

JosaAnalyzerService는 고레벨(L4~L6)에서 나타나는 조사 누락, 대치, 첨가 오류를 분석하기 위한 모듈이다. 한국어는 체언 뒤에 붙는 조사가 문장 의미와 문법 관계를 결정하므로, 단순 문자 단위 편집거리만으로는 “공룡이”를 “공룡”으로 읽은 오류와 다른 음운 오류를 명확히 구분하기 어렵다. 이를 보완하기 위해 원문과 STT 결과를 형태소 단위로 분해한 뒤 체언-조사 쌍을 정렬한다.

단계	처리 함수	구현 내용
1	<code>_extract(text)</code>	Mecab 품사 태깅 결과에서 NN*, NP, NR 등 체언과 J* 조사 태그를 결합하여 StemJosa(stem, josa) 리스트를 추출한다.
2	<code>_align(target, stt)</code>	원문과 STT의 StemJosa 시퀀스를 동적 계획법으로 정렬한다. 체언이 약간 다르게 인식되어도 Levenshtein 거리 기반 비용으로 같은 어절인지 판단한다.
3	<code>_detect(aligned)</code>	정렬 결과를 순회하면서 target_josa는 있으나 stt_josa가 없으면 DELETION, 서로 다르면 SUBSTITUTION, 원문에 없는 조사가 STT에 있으면 INSERTION으로 분류한다.
4	<code>get_top_event(events)</code>	Counter로 가장 자주 등장한 (kind, target_josa, stt_josa) 조합을 대표 오류로 선정하여 조사 퀴즈 생성에 전달한다.

위 알고리즘의 핵심은 조사 오류를 문자 차이가 아니라 체언 뒤 문법 요소의 차이로 해석한다는 점이다. 이 방식은 한국어의 조사 생략·대치 오류를 별도로 집계할 수 있게 하며, 이후 L4~L6 아동에게 조사 사용 연습 문장을 제공하는 기반 데이터가 된다.

6-3) 기능과 KoNLPy 모듈의 연관성

서비스 기능	관련 KoNLPy 모듈	역할
관심사 기반 동화 생성	InterestRefinerService(Okt)	STT 결과에서 동화 주제로 사용할 명사 키워드만 추출하고, 불용어·금지어를 제거하여 GPT 입력 품질을 높인다.
음성 기반 읽기 오류 패턴 분석	JosaAnalyzerService(Mecab)	원문과 STT 결과를 형태소 단위로 비교하여 조사 누락·대치·첨가 오류를 감지한다.
오류 유형 기반 퀴즈 생성	JosaAnalyzerService + QuizGeneratorService	빈도가 높은 조사 오류를 JosaEvent로 변환하여 조사 퀴즈 생성 프롬프트의 입력으로 사용한다.

KoNLPy 모듈은 생성형 AI의 보조 도구가 아니라, GPT가 사용할 입력 데이터의 품질을 제어하고 한국어 문법 단위의 오류를 구조화하는 전처리·분석 엔진이다. 따라서 GPT 모듈과 결합될 때 관심사 기반 동화 생성의 안전성, 조사 퀴즈 생성의 개인화 수준을 동시에 높인다.

3.3.7 Spring Security (JWT 기반 인증·인가)

토토리 백엔드는 iOS 모바일 앱과 Spring Boot 서버가 REST API로 통신하는 구조이므로, 매 요청마다 서버 세션을 유지하지 않는 JWT 기반 무상태 인증 방식을 사용한다. 다만 카카오 OAuth2 로그인 과정에서는 외부 인증 제공자와의 핸드셰이크를 위해 임시 세션이 필요하므로, SessionCreationPolicy는 IF_REQUIRED로 설정하였다. 일반 API 호출에서는 Access Token을 Authorization 헤더로 전달하고, Refresh Token은 Redis에 저장하여 재발급과 로그아웃을 서버 측에서 통제한다.

7-1) 인증·인가 모듈 구성

구현 모듈	역할	구현 상세
SecurityConfig	보안 필터 체인 구성	CSRF, HTTP Basic, Form Login을 비활성화하고 /api/auth/**, OAuth2 관련 경로는 permitAll로 설정한다. /api/parent/**는 PARENT, /api/child/**는 CHILD 권한을 요구하도록 매핑한다.
JwtTokenProvider	JWT 생성·검증	memberId를 subject로 설정하고, Access Token에는 role claim을 포함한다. 서명 검증, 만료 검증, 잔여 만료시간 계산을 담당한다.
JwtAuthenticationFilter	요청별 인증 처리	OncePerRequestFilter로 구현하여 요청마다 Authorization 헤더를 파싱하고, 유효한 토큰이면 SecurityContextHolder에 인증 객체를 등록한다.
CustomUserPrincipal	인증 사용자 표현	memberId와 Role(PARENT/CHILD)을 담아 컨트롤러와 서비스 계층에서 현재 사용자를 식별할 수 있게 한다.

AuthRedisService	토큰 상태 관리	RT:{memberId} 형식으로 Refresh Token을 저장하고, 로그아웃된 Access Token은 잔여 TTL 동안 블랙리스트에 등록한다.
------------------	----------	--

Spring Security가 단순히 로그인 기능만 담당하는 것이 아니라, 토큰 발급·검증·권한 분기·로그아웃 무효화를 각각 별도 모듈로 처리한다. 특히 아동과 보호자가 서로 다른 API 권한을 가지므로, JWT의 role claim과 SecurityConfig의 URL 권한 매핑이 서비스 구조와 직접 연결된다.

7-2) 토큰 정책 및 저장 방식

항목	Access Token	Refresh Token
유효기간	30분	14일
저장 위치	클라이언트가 Authorization: Bearer {token} 헤더로 전달	서버 Redis에 RT:{memberId} key로 저장
주요 Claim	subject=memberId, role=Role	subject=memberId
용도	일반 API 접근 인증 및 역할 기반 인가	Access Token 만료 시 재발급
무효화 방식	로그아웃 시 잔여 TTL만큼 Redis 블랙리스트 등록	로그아웃 시 Redis RT key 삭제

Access Token에만 role을 포함하고 Refresh Token에는 memberId만 포함한 이유는 Refresh Token의 역할을 재발급 권한 확인으로 제한하기 위해서이다. 실제 API 접근 권한은 짧은 수명의 Access Token에서 확인하고, 재발급 시에는 Redis에 저장된 Refresh Token과 회원 정보를 기반으로 새로운 Access Token을 발급한다.

7-3) JwtAuthenticationFilter 요청 처리 흐름

1. Authorization 헤더에서 Bearer 접두사를 제거하여 토큰 문자열을 추출한다.
2. AuthRedisService.isBlacklisted(token)으로 로그아웃된 Access Token인지 확인한다.
3. JwtTokenProvider.validateToken(token)으로 서명과 만료시간을 검증한다.
4. 토큰의 subject에서 memberId를, claim에서 role을 추출한다.
5. CustomUserPrincipal과 SimpleGrantedAuthority를 생성하여 UsernamePasswordAuthenticationToken을 구성한다.
6. SecurityContextHolder에 인증 객체를 저장한 뒤 다음 필터로 요청을 전달한다.
7. 검증 실패 시 authenticationEntryPoint 또는 accessDeniedHandler가 공통 JSON 오류 응답을 반환한다.

이 흐름을 통해 컨트롤러는 매번 토큰을 직접 파싱하지 않아도 되고, 서비스 계층에서는 인증된 memberId와 role을 기반으로 아동/보호자 기능을 분리할 수 있다.

7-4) 기능과 Security 모듈의 연관성

서비스 기능	관련 Security 모듈	역할
--------	----------------	----

회원가입/로그인/토큰 재발급	SecurityConfig + JwtTokenProvider + AuthRedisService	로그인 성공 시 Access/Refresh Token을 발급하고, Refresh Token을 Redis에 저장하여 재발급을 관리한다.
보호자/아동 역할 기반 API 분리	JwtAuthenticationFilter + CustomUserPrincipal	JWT의 role claim을 권한 객체로 변환하여 /api/parent/**와 /api/child/** 접근을 분리한다.
로그아웃	JwtTokenProvider + AuthRedisService	Access Token은 잔여 만료시간만큼 블랙리스트에 등록하고, Refresh Token은 Redis에서 삭제한다.

Security 모듈은 사용자 인증 자체뿐 아니라 토토리의 핵심 사용자 구조인 보호자/아동 역할 분리를 보장한다. 보호자는 리포트 조회와 자녀 연결 기능을 사용하고, 아동은 동화 읽기·퀴즈·출석 기능을 사용하므로, 역할 기반 인가는 기능 흐름의 전제 조건에 해당한다.

3.3.8 카카오 OAuth 2.0 소셜 로그인

카카오 로그인은 보호자 또는 아동이 별도의 아이디, 비밀번호를 직접 입력하지 않고 서비스에 진입할 수 있도록 구현한 소셜 로그인 기능이다. 본 서비스의 특징은 OAuth2 표준 흐름에 역할 사전 선택 단계를 결합했다는 점이다. 보호자와 아동은 가입 이후 접근 가능한 기능이 다르기 때문에, 카카오 인증 요청 전에 사용자가 선택한 Role(PARENT/CHILD)을 세션에 임시 저장하고, 콜백 처리 시 신규 회원의 역할로 반영한다.

8-1) 핵심 구현 모듈

구현 모듈	역할	구현 상세
CustomOAuth2UserService	카카오 사용자 정보 조회 및 회원 저장	DefaultOAuth2UserService를 호출하여 카카오 응답을 받은 뒤, providerId를 기준으로 기존 회원을 조회하거나 신규 회원을 생성한다.
CustomOAuth2User	OAuth2 인증 사용자 객체	memberId, role, attributes를 포함하여 인증 성공 이후 SuccessHandler와 SecurityContext에서 사용할 수 있게 한다.
OAuth2SuccessHandler	로그인 성공 후 JWT 발급	카카오 인증 성공 직후 Access Token과 Refresh Token을 발급하고, Refresh Token을 Redis에 저장한 뒤 앱 콜백 URL로 전달한다.
OAuth2Attribute	카카오 응답 매핑	provider, providerId, nickname 등 카카오 사용자 속성을 서비스 도메인에서 사용할 수 있는 형태로 변환한다.

Member / LoginType / MemberStat	도메인 저장	loginId는 KAKAO_{providerId}로 저장하고, 신규 회원 생성 시 학습 통계 누적을 위한 MemberStat도 함께 생성한다.
---------------------------------	--------	---

8-2) 역할 사전 선택 및 회원 등록 알고리즘

1. 클라이언트가 카카오 로그인 시작 전 보호자 또는 아동 역할을 선택한다.
2. Spring Boot 서버는 선택된 Role을 HttpSession의 SOCIAL_LOGIN_ROLE 속성에 임시 저장한다.
3. 사용자는 /oauth2/authorization/kakao 경로를 통해 카카오 인증 화면으로 이동한다.
4. 카카오 인증 콜백에서 CustomOAuth2UserService가 providerId와 nickname을 추출한다.
5. 기존 KAKAO_{providerId} 회원이 없으면 세션의 SOCIAL_LOGIN_ROLE을 읽어 신규 Member를 생성한다.
6. 신규 회원의 학습 통계 초기화를 위해 MemberStat을 함께 생성한다.
7. 회원 저장 후 finally 블록에서 SOCIAL_LOGIN_ROLE을 제거하여 다음 로그인에 영향을 주지 않게 한다.

```

Role role = (Role) session.getAttribute("SOCIAL_LOGIN_ROLE");
if (role == null) {
    throw new OAuth2AuthenticationException(
        new OAuth2Error("MISSING_ROLE_IN_SESSION")
    );
}

Member member = Member.builder()
    .loginId("KAKAO_" + providerId)
    .name(nickname)
    .password(null)
    .role(role)
    .loginType(LoginType.KAKAO)
    .build();

```

역할 정보를 세션에 임시 저장하는 방식은 OAuth2 provider가 토토리의 PARENT/CHILD 역할을 알 수 없기 때문에 필요하다. 단, 세션 값은 OAuth2 콜백 처리에만 사용하고 즉시 제거하여 잘못된 역할 정보가 다음 로그인에 재사용되지 않도록 하였다.

8-3) 인증 성공 후 JWT 발급 흐름

1. OAuth2SuccessHandler가 Authentication에서 CustomOAuth2User principal을 추출한다.
2. memberId와 role을 기반으로 JwtTokenProvider.createAccessToken(memberId, role)을 호출한다.
3. memberId만 포함한 Refresh Token을 생성하고 AuthRedisService.saveRefreshToken()으로 Redis에 저장한다.
4. 클라이언트가 토큰을 수신할 수 있도록 앱 콜백 또는 성공 리다이렉트 URL로 accessToken, refreshToken, role을 전달한다.
5. 이후 클라이언트는 일반 API 요청에서 Authorization 헤더에 Access Token을 포함하여 호출한다.

8-4) 기능과 카카오 모듈의 연관성

서비스 기능	관련 Kakao/Security 모듈	역할
보호자 1차 가입 및 자녀 등록 워크플로우	CustomOAuth2UserService + OAuth2SuccessHandler	보호자가 카카오 계정으로 가입한 뒤 JWT를 수신하고, 인증된 상태에서 자녀 프로필 등록 및 연결 API를 호출할 수 있게 한다.
아동 로그인 및 학습 기능 진입	CustomOAuth2User + JwtAuthenticationFilter	아동 role이 포함된 Access Token을 통해 /api/child/** 경로 접근을 허용한다.
역할 분기형 홈 화면 진입	OAuth2SuccessHandler + 클라이언트 라우팅	로그인 성공 시 role 값을 전달하여 앱에서 보호자 홈과 아동 홈으로 분기할 수 있게 한다.

카카오 로그인만 사용자의 진입 장벽을 낮추는 기능이지만, 토토리에서는 보호자와 아동의 역할 분기, 자녀 연결, 학습 통계 초기화까지 연결되는 시작점이다. 따라서 카카오 OAuth2 모듈은 인증 편의성뿐 아니라 서비스 도메인 흐름을 시작시키는 역할을 수행한다.

3.3.9 아키텍처 설계

안정적인 서비스 운영과 대용량 AI 트래픽 처리를 위해 다중 계층의 클라우드 인프라를 설계하였다.

- **AWS EC2 & Nginx**
 - 모바일 앱 및 웹 클라이언트의 모든 요청이 가장 먼저 도달하는 최초 진입점(API Gateway)이다.
 - EC2 인스턴스 내부에 구축된 Nginx가 리버스 프록시로 작동하며, URL 경로에 따라 트래픽을 동적으로 라우팅한다.
 - 일반적인 비즈니스 로직 및 데이터 조회 API(/api/*)는 동일한 EC2 인프라 내의 Spring Boot 기반 백엔드 서버(Port 8080)로 포워딩한다.
 - AI 추론 등 무거운 작업이 필요한 API(/ai/*)는 연산 자원이 분리된 GCP 서버로 포워딩하여 메인 서버의 부하를 방지한다.
- **GCP**
 - FastAPI 기반 AI 추론용 서버로, Whisper(STT), GPT(LLM), Stable Diffusion 등 무거운 AI 모델 연산을 전담하기 위해 분리 구축된 환경이다.
 - GCP 인스턴스 상에 Python FastAPI 서버를 배포하여, Nginx로부터 라우팅된 /ai/* 요청을 독립적으로 처리한다.
 - 이를 통해 대용량 오디오 파일 처리나 복잡한 알고리즘이 수행되더라도, Spring Boot 백엔드 서버의 응답 속도나 안정성에 영향을 주지 않는 마이크로서비스 구조를 확보하였다.
- **AWS RDS (MySQL)**
 - 서비스의 사용자 정보, 누적된 읽기 오류 통계, 동화책 학습 기록을 안전하게 영구 저장하는 관계형 데이터베이스이다.
 - Spring Boot 서버가 JPA를 통해 RDS에 연결되어 데이터를 조회하고 삽입한다.

- AI 서버(GCP)에서 도출된 읽기 오류 분석 결과 역시 Spring Boot를 거쳐 최종적으로 RDS에 정형 데이터로 동기화되어, 향후 아동의 성장 리포트 생성 및 퀴즈 출제를 위한 기반 데이터로 활용된다.

- **AWS S3**

- AI 생성 이미지 및 낭독 음성 등 대용량 미디어 파일을 안전하게 보관하고 제공하는 객체 스토리지이다.
- Stable Diffusion을 통해 실시간으로 생성된 동화책의 장면별 삽화 이미지 파일과, 아동의 낭독 오디오 파일을 S3 버킷에 업로드하여 영구 저장한다.
- Spring Boot 서버는 클라이언트가 해당 미디어 파일에 즉시 접근할 수 있도록 S3의 파일 URL을 반환한다.

3.4. 주요 기능의 구현

주요 기능	관련 SW 모듈	구현 흐름 요약
관심사 기반 동화 생성	MakeStoryBookView, BookService, FastApiStoryClient, StoryOrchestratorService, PromptBuilder, LLMStoryGeneratorService	아동 음성 입력 → STT/관심사 정제 → 제약조건 생성 → GPT 동화 생성 → 이미지/TTS 생성
음성 기반 읽기 오류 분석	stt_router, whisper_loader, alignment_utils, phoneme_analyzer	낭독 음성 전사 → 원문/STT 정렬 → 자모 단위 오류 분석 → 오류 패턴 저장
적응형 난이도 조절	BookService, ConstraintGeneratorService, LEVEL_CONSTRAINTS, PromptBuilder	최근 WCPM/레벨/취약 음운 조회 → 제약조건 생성 → GPT 프롬프트에 반영
오류 유형 기반 퀴즈	Redis, QuizGeneratorService, JosaAnalyzerService, GPT-5-mini	오류 통계 조회 → 대표 오류 선정 → 음운/조사 퀴즈 생성
보호자 리포트	ReportService, BookReadingRecord, Chart UI 등	누적 WCPM/정확도/오류 유형 기반 리포트 생성

3.4.1. 동화 생성 UI 플로우 구현 (MakeStoryBook)

1-1) 개요

동화 생성 화면은 아동이 음성으로 관심사를 말하면 AI가 맞춤 동화를 생성하는 전체 UX 플로우를 담당한다. 이 화면은 `GenerationStep` enum으로 정의된 4단계 상태 머신(State Machine)으로 구현되며, `MakeStoryBookViewModel`이 상태 전환과 비즈니스 로직을 관리한다.

1-2) 4단계 생성 플로우 (GenerationStep)

동화 생성 UI는 아래 4개의 단계로 구성된다. 각 단계는 `GenerationStep` enum의 case로 정의되며, 화면의 캐릭터 GIF 애니메이션, 채팅 메시지, 하단 버튼 타입이 단계별로 자동 변경된다.

단계	상태	화면 요소	캐릭터/메시지	사용자 액션
①	acorn	도토리 소모 확인	GIF: <code>Character_UsingAcorn</code> / "이야기를 만들기 위해선 도토리가 필요해!"	도토리 버튼 탭 → <code>acornCount</code> 차감 → <code>speak</code> 단계로 전환
②	speak	TTS 안내 재생	GIF: <code>Character_standard</code> / "오늘은 무슨 이야기를 해볼래?"	스피커 버튼 탭 → <code>listening</code> 단계로 전환
③	listening	음성 녹음 (마이크)	GIF: <code>Character_Listening</code> / 마이크 파동 애니메이션	마이크 탭(녹음 시작/종료) → 녹음 완료 시 <code>sprocessing</code> 으로 전환
④	processing	동화 생성 중 대기	GIF: <code>card_animation</code> / "동화 속 세상을 만드는 중..." + 자동 스크롤 팁 카드	서버 응답 수신 → <code>BookInfoView</code> 로 네비게이션

위 표에서 보듯이, 각 단계의 UI 요소(GIF 파일명, 채팅 메시지, 버튼 타입)는 `GenerationStep` enum에 정의된 계산 프로퍼티(`chatMessage`, `gifFileName`, `chatIcon`)를 통해 자동으로 결정된다. `ViewModel`의 `currentStep` 값이 변경되면 `SwiftUI`의 `@Published` 반응형 바인딩에 의해 화면이 자동으로 갱신된다.

1-3) 음성 녹음 및 서버 전송 플로우

`listening` 단계에서의 음성 녹음은 `AudioRecorderManager` 모듈이 담당한다. 처리 플로우는 다음과 같다.

- ① 마이크 버튼 첫 번째 탭: `AudioRecorderManager.requestPermission()` 호출 → iOS 마이크 권한 요청
 - iOS 17 이상: `AVAudioApplication.requestRecordPermission()` 사용
"동화 생성을 위해 아이의 목소리를 녹음합니다." 라는 권한 문구 등장으로 마이크가 왜 쓰이는지 설명
- ② 권한 획득 시: `AudioRecorderManager.startRecording()` 호출 → `AVAudioRecorder`로 녹음 시작
 - 녹음 설정: AAC 코덱(`kAudioFormatMPEG4AAC`), 12000Hz 샘플레이트, 모노 채널, 고품질
 - 임시 파일: `FileManager.default.directoryURL`에 `totori_record_{timestamp}.m4a`로 저장
- ③ 마이크 버튼 두 번째 탭(녹음 중지): `audioRecorder.stopRecording()` → 파일 URL 반환
- ④ `ViewModel`의 `requestBookGeneration(audioURL:)` 호출 → `BookService.makeBook(audioURL:)` 실행
 - Moya를 통해 `POST /api/books/make`에 `MultipartFormData(audio/m4a)`로 전송
- ⑤ 서버 응답(`BookGenerateResponseDTO`) 수신 → `navigateToBookInfo = true` → `BookInfoView`로 화면 전환

(4) AudioRecorderManager 구현 코드

```
class AudioRecorderManager: NSObject {
    private var audioRecorder: AVAudioRecorder?
    private var currentAudioURL: URL?

    func startRecording() {
        let session = AVAudioSession.sharedInstance()
        try session.setCategory(.playAndRecord, mode: .default,
                               options: [.defaultToSpeaker])
        try session.setActive(true)

        let tempDir = FileManager.default.temporaryDirectory
        let filename = "totori_record_\(Int(Date().timeIntervalSince1970)).m4a"
        let audioFileURL = tempDir.appendingPathComponent(filename)

        let settings: [String: Any] = [
            AVFormatIDKey: Int(kAudioFormatMPEG4AAC),
            AVSampleRateKey: 12000,
            AVNumberOfChannelsKey: 1,
            AVEncoderAudioQualityKey: AVAudioQuality.high.rawValue
        ]
        audioRecorder = try AVAudioRecorder(url: audioFileURL,
                                             settings: settings)
        audioRecorder?.record()
    }

    func stopRecording() -> URL? {
        audioRecorder?.stop()
        audioRecorder = nil
        return currentAudioURL
    }
}
```

```
}
```

(5) MakeStoryBookViewModel 핵심 코드

```
class MakeStoryBookViewModel: ObservableObject {
    @Published var currentStep: GenerationStep = .acorn
    @Published var isRecording: Bool = false
    @Published var navigateToBookInfo: Bool = false
    @Published var generatedBookData: BookGenerateResponseDTO? = nil

    private let audioRecorder = AudioRecorderManager()
    private let bookService = BookService()

    func performCenterAction() {
        switch currentStep {
        case .acorn:
            acornCount -= 1
            withAnimation { currentStep = .speak }
        case .speak:
            withAnimation { currentStep = .listening }
        case .listening:
            if !isRecording {
                audioRecorder.startRecording()
                isRecording = true
            } else {
                let fileURL = audioRecorder.stopRecording()
                currentStep = .processing
                requestBookGeneration(audioURL: fileURL!)
            }
        case .processing: break
        }
    }

    private func requestBookGeneration(audioURL: URL) {
        bookService.makeBook(audioURL: audioURL)
            .receive(on: DispatchQueue.main)
            .sink { ... }
            receiveValue: { [weak self] response in
                self?.generatedBookData = response
                self?.navigateToBookInfo = true
            }.store(in: &cancellables)
    }
}
```

3.4.2. 동화 읽기 UI 구현 (ReadStoryBook)

(1) 개요

동화 읽기 화면은 생성된 동화의 텍스트, 삽화 이미지, 음성을 통합하여 아동에게 인터랙티브한 읽기 경험을 제공하는 핵심 화면이다. ReadStoryBookViewModel이 서버 응답 데이터를 평탄화(Flatten)하여 페이지 단위로 렌더링하며, 마이크 녹음(따라 읽기)과 TTS 재생 기능을 제공한다.

(2) 데이터 평탄화 (Scene → DisplayPage)

서버 응답(BookGenerateResponseDTO)은 Scene 단위(pages[].sentences[])로 구조화되어 있으나, 동화 읽기 화면에서는 문장 1개 = 1페이지로 표시해야 한다. ReadStoryBookViewModel의 setUpData() 메서드가 이 변환을 수행한다.

변환 구조: BookPageResponseDTO[scene].sentences[문장들] → DisplayPage[globalIndex, imageUrl, text]

```
func setUpData(bookData: BookGenerateResponseDTO) {  
    //데이터 평탄화  
    var flatList: [DisplayPage] = []  
    var globalIdx = 0  
  
    // 페이지 순서 정렬  
    let sortedPages = bookData.pages.sorted {  
        $0.pageOrder < $1.pageOrder  
    }  
  
    for page in sortedPages {  
        for sentence in page.sentences {  
  
            let displayPage = DisplayPage(  
                globalIndex: globalIdx,  
                imageUrl: page.imageUrl,  
                text: sentence  
            )  
            flatList.append(displayPage)  
            globalIdx += 1  
        }  
    }  
    self.displayPages = flatList  
}
```

위 코드에서 핵심은, 같은 scene(BookPageResponseDTO)에 속하는 문장들은 모두 동일한 imageUrl을 공유한다는 점이다. 예를 들어 scene 1에 3개 문장이 있으면, 3개의 DisplayPage가 생성되지만 모두 scene 1의 삽화 이미지를 표시한다. 이로써 아동은 한 문장씩 넘기면서 읽되, 같은 장면의 이미지를 보며 이야기의 맥락을 유지할 수 있다.

(3) 동화 읽기 화면 UI 구성

ReadStoryBookView는 상단 이미지 영역과 하단 바텀시트 영역으로 분할된 2-Panel 레이아웃이다.

영역	UI 컴포넌트	구현 내용
상단 이미지	KFImage (Kingfisher)	currentDisplayPage.imageUrl을 Presigned URL로 로딩, retry(2회), fade(0.3s) 애니메이션, 메모리+디스크 캐싱
진행률 바	ProgressBar	progress = (currentIndex + 1) / displayPages.count, 실시간 갱신
텍스트 영역	Text + onTapGesture	현재 문장 표시, 탭하면 TTS 재생 토글 (배경색main20으로 변경)
하단 컨트롤	BookBottomControls	좌(이전)/중앙(마이크)/우(다음) 3버튼 구성, 마이크 녹음 시 파동 애니메이션
종료 모달	QuestionModal(.exitReading)	닫기 버튼 탭 시 확인 모달 표시 → 확인 시 MainView로 복귀

하단 BookBottomControls 컴포넌트의 중앙 버튼(CenterActionButton)은 마이크 녹음 상태에 따라 시각적으로 변화한다. 녹음 중(micRinging)일 때는 ForEach(0..<3)으로 3개의 동심원 파동 애니메이션이 재생되며, Animation.easeOut(duration: 3.0).repeatForever(autoreverses: false)로 무한 반복된다. 각 파동은 delay(index * 0.5)로 시차를 두어 물결 효과를 연출한다.

(4) 페이지 네비게이션 및 따라 읽기

동화 읽기의 페이지 전환 및 마이크 녹음(따라 읽기) 로직은 ReadStoryBookViewModel에서 관리한다.

- goNext(): currentIndex를 1 증가시키며, 마지막 페이지에서는 checkBadgeCompletion()을 호출하여 배지 획득 여부를 확인한다. 배지 획득 시 BadgeEarnedView로, 미획득 시 BookEndView로 네비게이션한다.
- goPrev(): currentIndex를 1 감소시킨다. 첫 페이지에서는 비활성화(isPrevEnabled = false).
- toggleMic(): 마이크 녹음을 시작/중지한다. 녹음 시작 시 TTS가 재생 중이면 자동 정지한다. 녹음 완료 시handleRecordedAudio(url:)로 후처리를 수행한다(현재 TODO: STT 서버 전송).
- toggleTTS(): 텍스트 영역을 탭하면 TTS 재생/정지를 토글한다. 재생 중에는 텍스트 배경색이 main20으로 변경되어 시각적 피드백을 제공한다.

3.4.3. 메인 화면 및 동화 목록 구현 (MainView)

(1) 개요

MainView는 앱의 중심 화면으로, 현재 읽고 있는 동화(FeaturedBookCard), 배지 진행 상황(BadgeCard), 전체 동화 목록(LazyVGrid)을 한 화면에 통합 제공한다. MainViewModel이 MainStatusService를 통해 서버에서 데이터를 조회하며, 동화 목록은 무한 스크롤(페이지네이션)로 구현하였다.

(2) 동화 목록 무한 스크롤 (페이지네이션)

동화 목록은 LazyVGrid(2열 그리드)로 렌더링되며, 마지막 아이템이 화면에 나타날 때(.onAppear) 다음 페이지를 자동 로딩하는 무한 스크롤 패턴을 구현하였다.

```
// MainView.swift - 무한 스크롤 구현
ForEach(viewModel.bookItems) { item in
    NavigationLink(destination: BookInfoView(bookData: ...)) {
        StoryBookView(type: item.type)
    }
    .onAppear {
        if item.id == viewModel.bookItems.last?.id {
            viewModel.fetchNextPage() // 마지막 아이템 도달 시 다음 페이지 로딩
        }
    }
}

// MainViewModel.swift - 페이지네이션 로직
func fetchNextPage() {
    guard !isLastPage else { return }
    fetchBookList(reset: false) // 기존 목록에 append
}
```

(3) 동화 상태별 UI 분기 (BookType)

동화 카드는 BookType enum을 통해 4가지 상태로 시각적으로 구분된다.

BookType	조건	UI 표현
create	새 동화 생성 버튼	보라색 텍스트 "이야기 시작하기", MakeStoryBookView로 네비게이션
unread	progress == 0	표지 이미지 표시, 핑크색 스파인(책등), 진행률 바 없음
reading	0 < progress < 1	표지 이미지 + 하단 진행률 ProgressBar 표시
finished	progress >= 1	표지 이미지 + 완독 배지 아이콘 표시

MainViewModel의 mapToBookItem() 메서드가 서버 응답(BookSummaryDTO)의 progress 값을 기반으로 BookType을 자동 결정한다. 이를 통해 동일한 StoryBookView 컴포넌트가 상태에 따라 다른 시각적 표현을 제공한다.

3.4.4. 네트워크 통신 계층 구현 (Moya + Combine)

(1) API 추상화 (TotoriAPI enum)

모든 서버 통신은 TotoriAPI enum으로 추상화된다. 각 case가 하나의 API 엔드포인트에 대응하며, path, method, task, headers, authorizationType을 프로토콜(BaseTargetType)로 구현한다.

API case	HTTP Method	경로	task (요청 형태)
generateBook(param)	POST	/api/books/generate	requestJSONEncodable (BookGenerateRequestDTO)
makeBook(audioURL)	POST	/api/books/make	uploadMultipart (audio/m4a MultipartFormData)
mainStatus	GET	/api/books/main-status	requestPlain
bookList(page, size)	GET	/api/books	requestParameters (URLEncoding.queryString)
attendance	POST	/api/child/attendance	requestPlain

특히 makeBook의 경우, 음성 파일(M4A)을 MultipartFormData로 업로드하는 구현이 핵심이다. audioURL에서 Data(contentsOf:)로 바이너리를 읽어 MultipartFormData(provider: .data, name: "audio", mimeType: "audio/m4a")로 변환한다. 인증이 필요한API(login, signUp, reissue 제외)에는 authorizationType = .bearer가 설정되어 TokenInterceptor가 자동으로 JWT 토큰을 헤더에 첨부한다.

(2) BaseService 공통 요청 처리

BaseService<Target: BaseTargetType>은 제네릭 서비스 클래스로, MoyaProvider를 래핑하여 공통적인 요청/응답 처리를 수행한다. Combine의 AnyPublisher<T, NetworkError>를 반환하며, 빈 응답 처리, JSON 디코딩, 에러 매핑을 자동화한다. TokenInterceptor는 Alamofire Session의 interceptor로 등록되어 401 응답 시 자동으로 /api/auth/reissue를 호출하여 토큰을 갱신한다.

3.4.5. 동화 생성 로직 구현

(1) 개요

동화 생성은 Totori 앱의 핵심 기능으로, 사용자(아동)가 말한 관심사를 기반으로 맞춤형 동화를 자동 생성한다. 이 기능은 두 가지 입력 경로를 지원한다: 텍스트 입력 방식(BookGenerateRequest.sttText)과 음성 입력 방식(MultipartFile audioFile)이다. 음성 입력의 경우FastAPI 서버에서 Whisper STT로 텍스트 변환까지 처리한다.

텍스트 입력 방식의 경우 stt 연결 전, 테스트 용으로 제작하였다.

동화 생성 파이프라인의 구현을 위해 다음 모듈들을 도입·구현하였다:

- FastAPI 측: StoryOrchestratorService (파이프라인 조율), InterestRefinerService (관심사 정제), ConstraintGeneratorService (제약조건 생성), prompt_builder (프롬프트 빌드), LLMStoryGeneratorService (GPT 호출)
- Spring Boot 측: BookService (전체 흐름 관리), FastApiStoryClient (FastAPI 호출), TtsService (음성 합성), StableDiffusionService (이미지 생성), PageImageAsyncService (비동기 이미지 처리)

(2) 동화 생성 전체 플로우

동화 생성의 전체 처리 흐름은 크게 4단계(관심사 정제 → 제약조건 생성 → GPT 동화 생성 → 멀티미디어 조합)로 구성된다. 아래 표는 단계별 상세 흐름을 요약한 것이며, 이후 각 단계에 대해 상세히 설명한다.

단계	처리 주체	모듈	처리 내용
①	iOS Swift	MakeStoryBookView	아동이 관심사를 음성으로 입력, '동화 만들기' 버튼 탭
②	Spring Boot	BookService	Member의 level, 최근 WCPM, 취약 발음 조회 후 FastAPI 요청 구성
③	Spring Boot → FastAPI	FastApiStoryClient	POST /ai/story/generate로 stt_text, level, recent_wcpm, weak_phonemes 전송
④	FastAPI	InterestRefinerService	KoNLPy(Okt) 형태소 분석 → 불용어/금지어 필터링 → 관심사 키워드 정제
⑤	FastAPI	ConstraintGeneratorService	level(L1~L6) 기반 제약조건 생성
⑥	FastAPI	prompt_builder	제약조건을 GPT System/User 프롬프트에 주입
⑦	FastAPI → OpenAI	LLMStoryGeneratorService	GPT-5.1 호출 → StoryResponse JSON 수신

⑧	FastAPI → Spring Boot	-	FastApiStoryResponse 반환
⑨	Spring Boot	BookService	Book, BookPage 엔티티 생성 + 공통 seed 생성
⑩	Spring Boot	PageImageAsyncService	@Async 이미지 병렬 생성 + S3 업로드
⑪	Spring Boot	TtsService	CompletableFuture TTS 병렬 생성 + S3 업로드
⑫	Spring Boot	BookRepository	최종 Book 엔티티 MySQL 저장
⑬	Spring Boot → iOS	-	BookGenerateResponse JSON 반환 → 동화 뷰어 렌더링

이하 각 단계의 상세한 처리 내용을 설명한다.

단계 ① iOS Swift 클라이언트 — 사용자 입력 및 음성 녹음

동화 생성은 MakeStoryBookView에서 시작된다. 이 화면은 GenerationStep enum으로 정의된 4단계 상태 머신(acorn → speak → listening → processing)으로 동작한다. 먼저 acorn 단계에서 도토리(앱 내 재화) 1개를 소모한다. 이어서 speak 단계에서 캐릭터가 "오늘은 무슨 이야기를 해볼래?"라는 안내 메시지를 표시하고, listening 단계로 진입하면 마이크 버튼이 활성화된다.

아동이 마이크 버튼을 탭하면 AudioRecorderManager가 iOS 마이크 권한을 요청한 뒤, AVAudioRecorder를 통해 녹음을 시작한다. 녹음 설정은 AAC 코덱(kAudioFormatMPEG4AAC), 12000Hz 샘플레이트, 모노 채널이며, FileManager.temporaryDirectory에 totori_record_{timestamp}.m4a 파일로 저장된다. 아동이 다시 마이크 버튼을 탭하면 녹음이 종료되고, processing 단계로 전환되면서 녹음된 M4A 파일이 서버로 전송된다.

서버 전송은 MakeStoryBookViewModel의 requestBookGeneration(audioURL:) 메서드에서 BookService.makeBook(audioURL:)을 호출하며, Moya의 TotoriAPI.makeBook case를 통해 POST /api/books/make 엔드포인트에 MultipartFormData(name: "audio", mimeType: "audio/m4a")로 업로드된다.

단계 ② Spring Boot — 사용자 데이터 조회 및 FastAPI 요청 구성

BookController가 POST /api/books/make 요청을 수신하면, @AuthenticationPrincipal에서 JWT 토큰 기반으로 현재 로그인된 memberId를 추출하고, BookService.generateBookFromVoice(memberId, audioFile)를 호출한다.

BookService는 먼저 MemberService.findById(memberId)를 호출하여 Member 엔티티를 로딩하고, Member의 level(MemberLevel enum: L1~L6) 값을 조회한다. 이어서 getLatestRecord(memberId)를 호출하여 해당 아동의 가장 최근 BookReadingRecord를 조회한다. BookReadingRecord에는 아동의 최근 WCPM(Words Correct Per Minute) 측정값과 mistakes(Map<String, Integer> 형태의 발음 오류 통계)가 저장되어 있다.

extractWeakPhonemes(latestRecord) 메서드는 mistakes Map에서 오류 횟수가 0보다 크고 빈도가 높은 상위 3개의 발음을 추출한다. 이 과정에서 Map.Entry를 값(오류 횟수) 기준으로 내림차순 정렬하고 .limit(3)으로 상위 3개만 선택한다.

```
// BookService.java - 취약 발음 추출
private List<String> extractWeakPhonemes(BookReadingRecord latestRecord) {
    if (latestRecord == null || latestRecord.getMistakes() == null) {
        return List.of();
    }
    return latestRecord.getMistakes().entrySet().stream()
        .filter(entry -> entry.getValue() > 0)
        .sorted(Map.Entry.<String, Integer>comparingByValue(
            Comparator.reverseOrder()))
        .limit(3)
        .map(Map.Entry::getKey)
        .toList();
}
```

음성 입력 경로(makeBook)의 경우, 음성 파일(MultipartFile)은 validateAudioFile()에서 파일 존재 여부, content-type(audio/*), 크기 제한(30MB)을 검증한 뒤, FastApiStoryClient.generateStoryFromAudio()를 통해 FastAPI 서버로 직접 전달된다. FastAPI 서버가 내부적으로 Whisper STT를 수행하여 음성을 텍스트로 변환한다.

단계 ③ Spring Boot → FastAPI — 서버 간 통신

FastApiStoryClient는 Spring Boot의 WebClient를 사용하여 FastAPI 서버와 통신한다. 음성 입력 경로에서는 POST /ai/story/make 엔드포인트로 MultipartFileData를 전송하며, 요청 본문에는 다음 필드가 포함된다:

- file: 아동의 음성 녹음 파일 (M4A binary)
- level: Member의 현재 레벨 문자열 (예: "L3")
- recent_wcpm: 최근 BookReadingRecord의 WCPM 값 (float, 없으면 0)
- weak_phonemes: 상위 3개 취약 발음 리스트 (JSON 문자열로 직렬화)

텍스트 입력 경로에서는 POST /ai/story/generate로 JSON 본문(stt_text, level, recent_wcpm, weak_phonemes)을 전송한다. 두 경로 모두 응답 타입은 FastApiStoryResponse(title, cover_image_prompt, pages[])로 동일하다.

단계 ④ FastAPI — 관심사 정제 (InterestRefinerService)

FastAPI의 story_router가 요청을 수신하면, 음성 입력 경로에서는 먼저 Whisper STT(whisper_loader.py)를 통해 음성을 한국어 텍스트로 변환한다. 이후 StoryOrchestratorService.run_pipeline()이 호출되어 파이프라인이 시작된다.

파이프라인의 첫 단계인 InterestRefinerService.refine(stt_text)는 아동이 말한 자연어 텍스트에서 동화 주제에 활용할 핵심 키워드를 추출한다. 처리 과정은 다음과 같다:

- ① KoNLPy의 Okt(Open Korean Text) 형태소 분석기로 명사(nouns)만 추출한다. 예: "나는 공룡이랑 우주 여행하고 싶어" → ["공룡", "우주", "여행"]
- ② 추출된 명사에서 FILLER_WORDS(불용어 31개: '음', '어', '그냥', '약간', '진짜', '완전', '막', '나', '좋아' 등)에 해당하는 단어를 제거한다.
- ③ BANNED_KEYWORDS(금지어 19개: '살인', '자살', '마약', '폭력' 관련 단어 등)에 해당하는 단어를 필터링하고, 감지 시 로그에 "유해 키워드 감지 및 제거됨"을 기록한다.
- ④ 중복을 제거(set 변환)하고 최종 관심사 키워드 리스트를 반환한다.

예를 들어, 아동이 "음... 나는 그냥 공룡이랑 로봇 이야기 좋아해"라고 말한 경우, 불용어 '음', '나', '그냥', '이야기', '좋아'가 제거되고, 최종적으로 ["공룡", "로봇"]이 동화 주제 키워드로 추출된다.

단계 ⑤ FastAPI — 제약조건 생성 (ConstraintGeneratorService)

ConstraintGeneratorService.generate()는 아동의 레벨(L1~L6)을 키로 LEVEL_CONSTRAINTS 딕셔너리를 조회하여, 해당 레벨에 맞는 제약조건(ConstraintResponse)을 생성한다. 각 레벨별 파라미터는 level_consts.py에 하드코딩되어 있으며, 핵심 파라미터는 다음과 같다:

- max_sentence_len: 문장 최대 어절 수 (L1: 4어절, L2: 5, L3: 7, L4: 13, L5: 15, L6: 17). GPT가 이 값을 초과하는 문장을 생성하지 않도록 프롬프트에 "절대 N어절을 넘지 말 것"으로 지시된다.
- vocab_level: 어휘 난이도 등급 (L1: "최하", L2: "하", L3: "중하", L4: "중", L5: "중상", L6: "상"). GPT 프롬프트에 직접 삽입되어 어휘 수준을 제한한다.
- scene_count / sentences_per_scene: 장면 수(6~8)와 장면당 문장 수(3~4). 이 두 값의 곱이 total_pages(18~24)가 되며, GPT에게 정확한 분량을 강제한다.
- target_wcpm: 레벨별 목표 WCPM (L1: 15, L6: 130). 단, recent_wcpm(실제 측정값)이 전달된 경우 이를 우선 사용하여 아동의 실제 읽기 속도에 맞춘 조정을 수행한다.

또한 단계 ④에서 추출된 관심사 키워드는 `theme_keywords`에, 단계 ②에서 추출된 취약 발음은 `focus_phonemes`에 설정된다. 최종 `ConstraintResponse` 객체에는 `level_name`, `age_group`, `max_sentence_len`, `vocab_level`, `total_pages`, `theme_keywords`, `focus_phonemes`, `scene_count`, `sentences_per_scene`, `adjusted_target_wcpm`의 10개 파라미터가 포함된다.

단계 ⑥ FastAPI — GPT 프롬프트 빌드 (prompt_builder)

`prompt_builder.build_story_prompt(constraints)` 함수는 `ConstraintResponse`의 모든 파라미터를 GPT의 System Prompt와 User Prompt에 주입한다.

System Prompt는 GPT에게 "난독증 아동을 위한 전문 한국어 동화 작가이자 특수교사" 역할을 부여하며, 핵심 규칙으로 모든 문장을 '-요'체로 작성할 것, 반복을 피하고 기승전결이 분명한 이야기를 만들 것, JSON 형식으로만 출력할 것을 지시한다.

User Prompt는 다음 [제약 조건]을 포함한다:

- 대상 연령(`age_group`), 동화 주제(`theme_keywords`를 쉼표로 연결), 어휘 난이도(`vocab_level`)
- 문장 길이: "한 페이지당 무조건 1개의 문장만 작성할 것. 한 문장당 절대 `{max_sentence_len}`어절을 넘지 말 것"
- L4 이상에서는 `min_sentence_len`도 적용: "한 문장당 최소 `{min_len}`어절 이상으로 쓸 것"
- 장면 구성: "반드시 정확히 `{scene_count}`개의 scene만 생성하고, 각 scene에는 반드시 `{sentences_per_scene}`개의 문장만 넣을 것"
- 취약 발음 연습: "`{focus_phonemes}`' 발음이 포함된 단어를 이야기가 어색하지 않은 선에서 자주 등장시킬 것"
- 캐릭터 외형 일관성: 모든 scene의 `image_prompt`에 주인공의 나이대, 성별, 머리색/스타일, 옷차림, 소지품, 화풍을 동일하게 유지할 것
- [대사 사용 규칙]: 전체 1~3개 페이지만 대사 허용, 대사는 큰따옴표 사용, '-요'체 예외 허용
- [출력 형식]: `title`, `cover_image_prompt`, `pages`[`{page_order, image_prompt, sentences[]}`] JSON 스키마

단계 ⑦ FastAPI → OpenAI — GPT-5.1 동화 생성

`LLMStoryGeneratorService.generate_story()`가 `AsyncOpenAI` 클라이언트를 통해 OpenAI Chat Completions API를 비동기 호출한다. 주요 설정은 다음과 같다:

- `model`: "gpt-5.1" — 한국어 동화 생성 품질과 JSON 출력 안정성이 가장 우수한 모델
- `messages`: [`{role: "system", content: system_prompt}`], [`{role: "user", content: user_prompt}`]
- `temperature`: 0.7 — 매번 다른 동화가 생성되도록 적당한 무작위성을 부여하되, 구조(JSON, 어절 수)는 준수
- `response_format`: `{"type": "json_object"}` — GPT가 반드시 JSON만 출력하도록 강제

GPT 응답의 content 문자열은 Pydantic의 StoryResponse.model_validate_json()을 통해 2중 검증된다. StoryResponse 스키마에는 title(str), cover_image_prompt(str), pages(List[Pages])가 정의되어 있으며, Pages에는 page_order(int), image_prompt(str), sentences(List[str])가 포함된다. GPT가 이 스키마를 위반하면 ValidationError가 발생하여 "LLM이 지정한 동화 JSON 형식을 지키지 않았습니다"라는 명확한 에러 메시지가 반환된다.

단계 ⑧ FastAPI → Spring Boot — 동화 데이터 반환

FastAPI의 StoryResponse 객체가 JSON으로 직렬화되어 HTTP 응답으로 반환된다. Spring Boot의 FastApiStoryClient는 이를FastApiStoryResponse record(title, coverImagePrompt, List<FastApiPageResponse> pages)로 역직렬화한다. 각FastApiPageResponse에는 pageOrder(int), imagePrompt(String), sentences(List<String>)가 포함된다. 이 데이터는 이후 단계에서 엔티티 생성, 이미지 생성, TTS 생성의 입력값으로 사용된다.

단계 ⑨ Spring Boot — 엔티티 생성 및 이미지 seed 할당

BookService.buildAndSaveBook()에서 FastApiStoryResponse를 기반으로 Book 엔티티와 BookPage 엔티티를 생성한다.

Book.of(member, fastApiResponse)는 Member(소유자), title, coverImagePrompt, totalPages(pages 배열 크기)를 설정하며, coverImageUrl은 아직 null로 초기화된다(이미지 생성 후 업데이트). 각 BookPage.of(book, pageResponse)는 pageOrder, imagePrompt를 설정하고, sentences 필드에는 각 문장 텍스트를 SentenceData.ofText(text)로 변환하여 JSON 타입 컬럼(@JdbcTypeCode(SqlTypes.JSON))에 저장한다. SentenceData는 text 외에 audioS3Key(TTS 파일명)와 durationMs(재생 시간)를 포함하지만, 이 시점에서는 모두 null이다.

또한 이 단계에서 ThreadLocalRandom.current().nextInt(10000000)으로 bookSeed(long 타입)를 생성한다. 이 seed는 단계 ⑩에서 모든 페이지 이미지 생성 시 동일하게 전달되어, Stable Diffusion이 같은 책 내에서 일관된 화풍과 캐릭터 외형을 유지하도록 한다.

단계 ⑩ Spring Boot — 삽화 이미지 비동기 병렬 생성

BookService는 표지 이미지(coverImagePrompt)와 각 페이지 이미지(page.imagePrompt)를 PageImageAsyncService를 통해 병렬로 생성한다. PageImageAsyncService는 @Async("imageExecutor") 어노테이션이 적용된 비동기 서비스로, AsyncConfig에서 정의한 ThreadPoolTaskExecutor(corePoolSize=2, maxPoolSize=3, queueCapacity=10)에서 실행된다.

각 이미지 생성의 내부 처리는 다음과 같다:

- ① `StableDiffusionService.generateImage(prompt, bookSeed)` 호출
 - prompt 앞에 "cute children's book illustration, watercolor style, " 접두어 자동 추가
 - Stability AI REST API(POST <https://api.stability.ai/v2beta/stable-image/generate/sd3>)에 `model="sd3.5-flash", aspect-ratio="4:6", output_format="png", seed=bookSeed` 파라미터로 요청
 - 응답 JSON에서 Base64 인코딩된 이미지 문자열을 추출하고, `Base64.getDecoder().decode()`로 byte[]로 변환
- ② `S3ImageStorageService.uploadImage(imageBytes, fileName)` 호출
 - AWS S3에 "bookImages/{UUID}_page_{N}.png" 키로 업로드 (content-type: image/png)
 - 업로드 완료 후 파일명(fileName)을 반환
- ③ `CompletableFuture.completedFuture(imageUrl)` 반환 → `BookPage.updateImageUrl(imageUrl)`로 엔티티에 이미지URL 기록

BookService는 표지 + 모든 페이지의 CompletableFuture를 `List<CompletableFuture<Void>>`에 수집하고, `CompletableFuture.allOf(futures.toArray(...)).join()`으로 전체 이미지 생성이 완료될 때까지 대기한다. 이 병렬 처리를 통해, 6~8개 이미지를 순차 생성 시 약 60~80초 걸리는 작업이 병렬 처리로 약 20~30초로 단축된다.

단계 ⑪ Spring Boot — TTS 음성 비동기 병렬 생성

이미지 생성 완료 후, Book 엔티티를 먼저 DB에 저장(`bookRepository.save`)한 뒤

`TtsService.generateAllAudios(savedBook)`를 호출한다. TTS 생성은 이미지 생성과 독립적인 별도의 병렬 처리 메커니즘을 사용한다.

TtsService는 `Executors.newFixedThreadPool(parallelism)`으로 고정 크기 스레드 풀(기본 3개 스레드, `application.yml`의 `tts.parallelism`으로 설정 가능)을 생성한다. 처리 과정은 다음과 같다:

- ① Book의 모든 BookPage를 순회하고, 각 페이지의 `List<SentenceData>`를 순회한다.
- ② 각 문장에 대해 `sentence.hasAudio()`를 확인하여 이미 음성이 생성된 문장은 건너뛴다(중복 호출 방지).
- ③ `CompletableFuture.runAsync(() -> { ... }, executor)`로 비동기 태스크를 생성한다:
 - `ElevenLabsClient.synthesize(sentence.getText())` 호출 → ElevenLabs API에 `POST /text-to-speech/{voiceId}` 전송
 - API 요청 본문: `{text, model_id: "eleven_multilingual_v2", voice_settings: {stability: 0.5, similarity_boost: 0.75}}`
 - 응답으로 MP3 byte[]를 수신
 - `S3AudioStorageService.uploadAudio(audio, "{bookId}_{pageOrder}_{sentenceIdx}.mp3")` → S3에 "bookAudios/" 경로로 업로드

- `sentence.updateAudio(fileName, null)`로 `SentenceData`의 `audioS3Key`에 파일명 기록
- ④ 개별 문장의 TTS 생성이 실패해도 `catch` 블록에서 로그만 기록하고 나머지 문장은 정상 처리한다(장애 격리).
- ⑤ `CompletableFuture.allOf(...).join()`으로 모든 문장 TTS 완료를 대기한 뒤, `executor.shutdown()`으로 스레드 풀을 종료한다.

TTS 전체가 실패하더라도 `BookService`의 `catch` 블록에서 예외를 잡아 로그만 기록하므로, 아동은 텍스트와 이미지만으로 동화를 읽을 수 있다(Graceful Degradation).

단계 ⑫ Spring Boot — MySQL 영속화

TTS 생성까지 완료된 `Book` 엔티티를 `bookRepository.save(savedBook)`으로 MySQL에 최종 저장한다. 이 시점의 `Book` 엔티티에는 다음 데이터가 모두 포함되어 있다:

- `Book`: `id`, `member(FK)`, `title`, `coverImageUrl(S3 파일명)`, `coverImagePrompt`, `totalPages`
- `BookPage (CascadeType.ALL)`: `id`, `book(FK)`, `pageOrder`, `imageUrl(S3 파일명)`, `imagePrompt`, `sentences(JSON)`
- `SentenceData (JSON 컬럼 내)`: `text(동화 문장)`, `audioS3Key(MP3 파일명)`, `durationMs`

`Book`과 `BookPage`는 `@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)` 관계로 설정되어, `Book` 저장 시 하위 `BookPage`들도 함께 저장된다. `sentences` 필드는 `@JdbcTypeCode(SqlTypes.JSON)`으로 선언되어 MySQL의 JSON 컬럼 타입으로 저장되며, `SentenceData` 리스트가 자동으로 직렬화/역직렬화된다.

단계 ⑬ Spring Boot → iOS — 최종 응답 반환 및 동화 뷰어 렌더링

`BookService`는 저장된 `Book` 엔티티를 기반으로 `BookGenerateResponse`를 구성한다. 이 과정에서 S3에 저장된 이미지·음성 파일에 대해 Presigned URL을 생성한다.

- 표지 이미지: `S3ImageStorageService.getPresignedUrl("bookImages", coverImageUrl) → 60분 유효 Presigned URL` 발급
- 각 페이지 이미지: 동일하게 `S3ImageStorageService.getPresignedUrl()`로 Presigned URL 생성
- 각 문장 음성: `S3AudioStorageService.getPresignedUrl("bookAudios", audioS3Key)`로 Presigned URL 생성

최종 `BookGenerateResponse`는 `bookId`, `title`, `totalPages`, `coverImagePrompt`, `coverImageUrl(Presigned)`, `pages[](pageId, pageOrder, sentences[text, audioUrl], imageUrl(Presigned))` 구조의 JSON으로 클라이언트에 반환된다.

iOS Swift 클라이언트에서는 MakeStoryBookViewModel이 응답을 BookGenerateResponseDTO로 디코딩하고, navigateToBookInfo = true를 설정하여 BookInfoView로 화면을 전환한다. BookInfoView에서 '이야기 읽기' 버튼을 탭하면ReadStoryBookView로 이동하여, 생성된 동화의 삽화 이미지(Kingfisher로 Presigned URL 로딩), 텍스트, TTS 음성을 통합한 동화 읽기 경험을 제공한다.

(3) 관심사 정제 모듈 (InterestRefinerService)

아동이 입력한 자연어 텍스트에서 동화 주제에 활용할 핵심 키워드를 추출하는 모듈이다. KoNLPy의 Okt(Open Korean Text) 형태소 분석기를 사용하여 명사만 추출하고, 미리 정의된 FILLER_WORDS(불용어: '음', '어', '그냥', '약간' 등 31개)와BANNED_KEYWORDS(금지어: 폭력/성적 표현 등 19개)를 필터링한다. 중복 제거 후 최종 관심사 키워드 리스트를 반환한다.

```
class InterestRefinerService:
    def __init__(self):
        self.okt = Okt()

    def refine(self, text: str) -> List[str]:
        nouns = self.okt.nouns(text)    # 명사 추출
        refine_keywords = []
        for noun in nouns:
            if noun in FILLER_WORDS: continue    # 불용어 제거
            if not self._is_safe_word(noun): continue    # 금지어 제거
            refine_keywords.append(noun)
        return list(set(refine_keywords))    # 중복 제거
```

(4) GPT 프롬프트 설계 상세 (prompt_builder)

prompt_builder.py는 ConstraintResponse 객체를 받아 GPT System Prompt와 User Prompt를 생성하는 함수이다. 프롬프트에는 다음 요소가 포함된다.

- System Prompt: "난독증 아동을 위한 전문 한국어 동화 작가이자 특수교사" 역할 부여, 모든 문장을 '-요'체로 작성하라는 문체 지시, JSON 출력 형식 강제
- User Prompt [제약 조건]: 대상 연령(age_group), 어휘 난이도(vocab_level), 문장 최대 어절 수(max_sentence_len), 장면 수(scene_count), 장면당 문장 수(sentences_per_scene), 총 페이지 수(total_pages), 취약 발음 연습 단어 반복 노출, 캐릭터 외형 일관성 규칙
- User Prompt [대사 사용 규칙]: 전체에서 1~3개 페이지만 대사 허용, 대사는 큰따옴표 사용, '-요'체 예외 허용
- User Prompt [출력 형식]: title, cover_image_prompt, pages[] (page_order, image_prompt, sentences[]) 구조의 JSON 스키마 명시

특히 취약 발음(focus_phonemes) 파라미터를 통해, 아동이 자주 틀리는 발음이 포함된 단어를 동화 내에 자연스럽게 반복 노출시키도록GPT에 지시하는 것이 핵심 교육적 설계이다.

3.4.6. 자동 난이도 조절 기능 구현

(1) 개요

자동 난이도 조절 기능은 아동의 읽기 레벨(L1~L6)과 최근 WCPM(Words Correct Per Minute) 측정값을 기반으로, GPT 프롬프트의 핵심 파라미터를 동적으로 조절하는 기능이다. 이 기능의 구현을 위해 FastAPI 측의 ConstraintGeneratorService 모듈과 LEVEL_CONSTRAINTS 상수 테이블, 그리고 Spring Boot 측의 MemberLevel 엔티티, BookService의 리딩 기록 조회 로직을 구현하였다.

(2) 레벨별 제약조건 정의 (LEVEL_CONSTRAINTS)

난이도는 L1~L6까지 6단계로 정의되어 있으며, 각 레벨별로 목표 WCPM, 문장 최대 어절 수, 어휘 난이도, 장면 수, 장면당 문장 수, 총 페이지 수가 결정된다. 아래 표는 level_consts.py에 정의된 실제 값이다.

레벨	대상 연령	목표WCPM	최대 어절수	어휘 수준	장면 수	장면당 문장수	총 페이지
L1	3~4세	15	4	최하	6	3	18
L2	5~6세	30	5	하	6	3	18
L3	7~8세	50	7	중하	6	3	18
L4	9~10세	80	13	중	9	3	27
L5	11~12세	110	15	중상	9	3	27
L6	13세	130	17	상	9	3	27

(3) 난이도 조절 처리 플로우

자동 난이도 조절의 전체 처리 흐름은 다음과 같다.

- ① Spring Boot의 BookService에서 Member 엔티티의 level(L1~L6 enum) 조회
- ② 동일 BookService에서 BookReadingRecord를 조회하여 최근 WCPM 값과 취약 발음(mistakes Map)을 추출
 - extractWeakPhonemes(): mistakes Map에서 오류 횟수가 높은 상위 3개 발음을 추출
- ③ FastApiGenerateStoryRequest에 level, recent_wcpm, weak_phonemes를 담아 FastAPI로 전송
- ④ FastAPI의 ConstraintGeneratorService에서 level 문자열로 LEVEL_CONSTRAINTS 딕셔너리 조회
 - recent_wcpm이 전달되면 해당 값을 adjusted_target_wcpm으로 설정 (아동의 실제 수준 반영)
 - weak_phonemes가 전달되면 focus_phonemes로 설정

- ⑤ 생성된 ConstraintResponse가 prompt_builder에 전달되어, 각 파라미터가 GPT 프롬프트에 동적으로 주입됨
 - 예: L1 → "한 문장당 절대 4어절을 넘지 말 것", "어휘 난이도: 최하"
 - 예: L5 → "한 문장당 최소 7어절 이상", "한 문장당 절대 15어절을 넘지 말 것", "어휘 난이도: 중상"

(4) ConstraintGeneratorService 구현 코드

```
class ConstraintGeneratorService:
    def generate(self, request: ProfileRequest) -> ConstraintResponse:
        base_consts = LEVEL_CONSTRAINTS[request.level]

        # WCPM: 최근 측정값이 있으면 사용, 없으면 레벨 기본값
        final_wcpm = request.recent_wcpm or base_consts["target_wcpm"]

        # 취약 발음: 전달된 값이 있으면 사용
        final_phonemes = request.weak_phonemes or []

        return ConstraintResponse(
            level_name=request.level,
            age_group=base_consts["age_group"],
            max_sentence_len=base_consts["max_sentence_len"],
            vocab_level=base_consts["vocab_level"],
            total_pages=base_consts["total_pages"],
            theme_keywords=request.interests,
            focus_phonemes=final_phonemes,
            scene_count=base_consts["scene_count"],
            sentences_per_scene=base_consts["sentences_per_scene"],
            adjusted_target_wcpm=final_wcpm
        )
```

(5) 난이도와 기능·모듈의 연관성

자동 난이도 조절 기능은 여러 SW 모듈에 걸쳐 동작한다. 아래 표는 기능과 모듈 간의 연관성을 정리한 것이다.

기능 요소	담당 모듈	역할 및 연관성
레벨 저장/조회	Member 엔티티 (Spring Boot)	MemberLevel enum(L1~L6)으로 아동의 현재 레벨을 영속적으로 관리
WCPM/취약발음 추출	BookService (Spring Boot)	BookReadingRecord에서 최근 wcpm 값과mistakes Map을 조회하여 상위 3개 취약 발음 추출
제약조건 매핑	ConstraintGeneratorService (FastAPI)	LEVEL_CONSTRAINTS 딕셔너리에서 레벨별 어절수, 장면수, 어휘 수준 등 파라미터 조회
프롬프트 주입	prompt_builder (FastAPI)	ConstraintResponse의 모든 파라미터를 GPT System/User Prompt에 동적으로 삽입

동화 텍스트 생성	LLMStoryGeneratorService (FastAPI)	난이도가 반영된 프롬프트로 GPT-5.1를 호출하여 레벨에 맞는 동화 생성
-----------	------------------------------------	---

위 표에서 보듯이, 난이도 조절 기능은 단일 모듈이 아니라 Spring Boot의 데이터 조회 계층과 FastAPI의 AI 처리 계층이 협력하는 크로스-서버 기능이다. Member의 level 정보와 BookReadingRecord의 학습 이력이 Spring Boot에서 추출되어 FastAPI로 전달되고, FastAPI에서 이를 GPT 프롬프트에 반영하여 최종적으로 아동 수준에 맞는 동화가 생성된다.

3.4.7 음성 기반 읽기 오류 패턴 분석

아동의 음성 전사 결과(STT)와 원본 동화 텍스트 간의 차이를 분석하기 위해, 단순 문자열 비교가 아닌 동적 계획법(DP) 기반의 형태소 및 음운 정렬 알고리즘인 편집거리 알고리즘을 사용하여 구현하였다.

7-1) 모듈 구성 및 역할

구현 모듈	주요 역할	상세
alignment_utils.py	단어/자소 단위 정렬	Levenshtein Distance(편집 거리) 알고리즘을 활용하여 길이가 다른 두 텍스트 배열을 1:1로 매칭하고 빈 공간에 -(공백)을 삽입한다.
phoneme_analyzer.py	음운 오류 분류	정렬된 데이터를 초/중/종성 자모 단위로 분해하고, 대치·탈락·첨가의 구체적인 오류 유형을 판별하여 추출한다.

7-2) 동적 계획법(DP) 기반 텍스트 정렬 로직 (alignment_utils.py)

아동이 단어를 빼먹고 읽거나(길이 감소) 더듬어 추가로 읽을 경우(길이 증가), 두 배열의 인덱스가 어긋난다. 이를 교정하기 위해 get_alignment 함수에서 DP 매트릭스를 생성하고 역추적하는 조건을 세밀하게 구현하였다.

```
# DP 매트릭스 역추적 (Traceback) 로직
while i > 0 or j > 0:
    if i > 0 and j > 0 and original[i - 1] == stt[j - 1]:
        # 기준 1: 두 값이 같으면 오류 없음 (그냥 통과)
        align1.append(original[i - 1])
        align2.append(stt[j - 1])
        i -= 1; j -= 1
    elif i > 0 and j > 0 and dp[i][j] == dp[i - 1][j - 1] + 1:
        # 기준 2: 대각선 값에서 +1 증가했으면 '대치(Substitution)' 발생
        align1.append(original[i - 1])
        align2.append(stt[j - 1])
        i -= 1; j -= 1
    elif i > 0 and dp[i][j] == dp[i - 1][j] + 1:
        # 기준 3: 위쪽 값에서 +1 증가했으면 원본에는 있으나 STT에 없는 '탈락/누락'
        align1.append(original[i - 1])
        align2.append("-") # STT 쪽에 공백(-) 삽입
```

```

        i -= 1
    else:
        # 기준 4: 왼쪽 값에서 +1 증가했으면 STT에 잘못 추가된 '첨가'
        align1.append("-") # 원본 쪽에 공백(-) 삽입
        align2.append(stt[j - 1])
        j -= 1

```

위 로직에서 if-elif-else의 기준점은 DP 매트릭스의 누적 비용 연산 결과이다. 현재 셀(dp[i][j])이 어느 방향(대각선, 위, 왼쪽)에서 최소 비용으로 넘어왔는지를 역추적하여, 아동이 문장 내 어느 위치에서 단어를 빠뜨렸는지, 또는 잘못 읽었는지를 정확하게 정렬한다.

7-3) 자모 단위 정밀 오류 판별 알고리즘 (phoneme_analyzer.py)

정렬된 단어 쌍 중 내용이 불일치하는 단어는 PhonemeAnalyzerService로 넘겨져 초/중/종성 자모 단위로 분해(j2hcj(h2j(text)))된다. 이후 분해된 자모 배열을 다시 위의 DP 로직으로 정렬하고, _get_errors 함수에서 최종 오류 패턴을 텍스트화한다.

```

def _get_errors(self, a1: list, a2: list) -> list[str]:
    errors = []
    for c1, c2 in zip(a1, a2):
        if c1 != c2: # 원본 자소(c1)와 STT 자소(c2)가 다를 때
            if c2 == "-":
                # 기준 1: STT 배열(c2)에 '-'가 있다면 아이가 발음하지 않은 것
                errors.append(f"{c1} 탈락")
            elif c1 == "-":
                # 기준 2: 원본 배열(c1)에 '-'가 있다면 아이가 불필요한 소리를 낸 것
                errors.append(f"{c2} 첨가")
            else:
                # 기준 3: 둘 다 글자가 존재하는데 다르다면 잘못 발음한 것
                errors.append(f"{c1} -> {c2} 대체")
    return errors

```

위 코드는 c1(원본 자소)과 c2(낭독 자소)의 비교 과정에서 하이픈(-)의 위치를 기준으로 삼는다. c2가 -이면 아동이 받침 등을 생략한 "탈락" 오류로, c1이 -이면 없는 발음을 추가한 "첨가" 오류로, 둘 다 문자가 있다면 "대체" 오류로 판별하여 ["ㄱ 탈락", "ㅏ -> ㅓ 대체"] 형태의 정밀한 오류 리스트를 생성한다.

7-4) 기능과 모듈의 연관성

이러한 읽기 오류 분석 과정은 본 서비스의 핵심 데이터 수집 역할을 한다.

1. stt_router가 오디오를 수신하여 whisper로 텍스트를 추출하면,
2. alignment_utils가 단어 단위 위치를 찾아내고,
3. phoneme_analyzer가 위 코드의 if 분기를 통해 정확히 어떤 자모에서 "탈락/첨가/대체"가 일어났는지 확인한다.

4. 최종적으로 `get_top_error()`를 통해 Counter 모듈로 가장 많이 누적된 오류(예: 'ㄱ 탈락')를 선정하며, 이 결과는 다음 단계인 "GPT 기반 맞춤형 음운 퀴즈 생성"의 핵심 제약 조건(Prompt Parameter)으로 즉시 연계된다.

3.4.8 퀴즈 생성

퀴즈 생성 기능은 1차 보고서의 주요 기능 리스트 중 “오류 유형 기반 맞춤형 반복 학습(퀴즈)”에 해당한다. 이 기능은 아동이 동화를 낭독한 뒤 발생한 오류를 단순히 기록하는 데서 끝나지 않고, 가장 자주 나타난 오류 유형을 즉시 학습 문항으로 변환하여 반복 연습 기회를 제공하는 것을 목표로 한다.

8-1) 전체 데이터 흐름

[iOS Client]

→ [Spring Boot BookController / BookService]

→ [FastApiStoryClient]

→ [FastAPI ReadingService]

→ [Whisper STT + PhonemeAnalyzerService + JosaAnalyzerService]

→ [Redis: reading_errors:{child_id}:{book_id}]

→ [QuizRouter]

→ [QuizGeneratorService + GPT-5-mini]

→ [QuizResponse(quiz_items[])]

위 흐름에서 Spring Boot는 모바일 클라이언트와 AI 서버 사이의 진입점 역할을 수행하고, FastAPI는 STT·오류 분석·퀴즈 생성을 담당한다. Redis는 동일한 책을 읽는 동안 발생한 문장별 오류를 짧은 시간 동안 누적 저장하는 세션 캐시로 사용된다.

8-2) 단계별 구현 흐름

단계	호출/모듈	입력	처리 내용	출력
1. 낭독 오디오 업로드	POST /api/books/{bookId}/reading/{sentenceNum}	문장 오디오, 원문, childId, bookId	클라이언트가 문장 단위 낭독 오디오를 Spring Boot로 전송한다.	multipart 요청
2. AI 서버 전달	FastApiStoryClient.analyzeReading()	multipart/form-data	Spring Boot가 FastAPI /ai/reading/anal	FastAPI 분석 요청

			alyze로 오디오와 원문을 전달한다.	
3. STT 전사	transcribe_with_timestamps(audio_path, "raw")	오디오 파일	Whisper가 자동 정규화를 최소화한 raw preset으로 아동 발화를 전사하고 타임스탬프를 추출한다.	전사 텍스트, segments
4. 오류 분석	PhonemeAnalyzerService + JosaAnalyzerService	원문, STT 결과	음운 단위 대치·생략·삽입 오류와 조사 단위 오류를 분리 추출한다.	error dict 리스트
5. WCPM 계산	ReadingService._calc_wcpm()	segments, 정답 어절 수	읽기 시간과 정확히 읽은 단어 수를 이용해 WCPM을 계산한다.	wcpm
6. 오류 저장	Redis RPush + EXPIRE	childId, bookId, errors	reading_errors:{child_id}:{book_id} key에 오류를 누적하고 TTL 3시간을 설정한다.	세션 오류 캐시
7. 퀴즈 요청	POST /ai/quiz/generate	child_id, book_id, level	레벨 유효성을 확인하고 L1~L3은 음운 퀴즈, L4~L6은 조사 퀴즈로 분기한다.	퀴즈 생성 트리거
8. 대표 오류 추출	get_top_phoneme_error / get_top_josa_error	Redis 오류 리스트	Counter로 가장 빈도 높은 오류 패턴을 선정한다.	target_word 또는 JosaEvent
9. LLM 문항 생성	QuizGeneratorService	대표 오류	GPT-5-mini를 호출하여 단어 또는 문장 후보 4개를 생성하고 중복을 제거한다.	QuizResponse

8-3) 레벨별 분기 로직

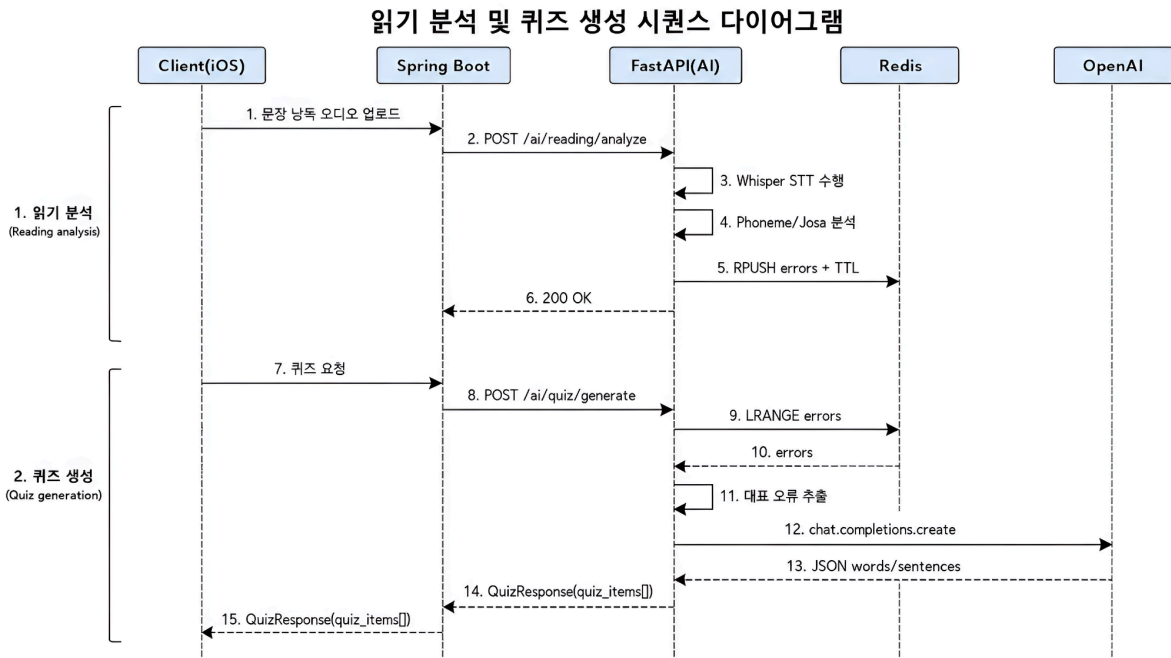
아동 레벨	대표 오류 추출 방식	퀴즈 생성 방식	학습 의도
L1~L3	get_top_phoneme_error(errors)	generate_quiz_words(target_word, error_pattern)	받침 탈락, 자음/모음 대치, 첨가 등 기초 음운 오류를 단어 단위로 반복 연습한다.
L4~L6	get_top_josa_error(errors)	generate_josa_quiz(JosaEvent)	문장 읽기에서 발생하는 조사 누락·대치·첨가를 짧은 문장 단위로 연습한다.

레벨별 분기 기준을 둔 이유는 저학년 또는 낮은 읽기 수준의 아동에게는 자모와 음절 단위 반복이 더 직접적이고, 높은 레벨의 아동에게는 문장 속 조사 사용 오류가 읽기 정확도에 더 큰 영향을 주기 때문이다. 따라서 동일한 퀴즈 기능이라도 아동 레벨에 따라 문제 단위와 프롬프트 전략이 달라진다.

8-4) 예외 처리 및 클라이언트 응답

- Redis에 분석 결과가 없으면 “분석된 오류가 없습니다.” 메시지와 함께 404를 반환한다. 이는 아동이 완벽하게 읽었거나 아직 낭독 분석이 완료되지 않은 상태를 구분하기 위함이다.
- LLM 응답이 JSON 형식을 지키지 않거나 필수 key(words/sentences)가 없으면 서버 내부 예외로 처리하고, 클라이언트에는 재시도 가능한 오류로 전달한다.
- 중복 문항은 _dedupe() 후처리에서 제거한다. 음운 퀴즈는 첫 단어를 실제 오답 단어로 유지하여 학습 연속성을 보장한다.
- 최종 응답은 QuizResponse{quiz_items: List[str]} 형태로 통일하여 iOS 클라이언트가 음운 퀴즈와 조사 퀴즈를 동일한 UI 컴포넌트로 렌더링할 수 있게 한다.

8-5) 시퀀스 다이어그램



이 시퀀스는 퀴즈 생성이 동화 낭독 직후의 분석 데이터를 활용한다는 점을 명확히 보여준다. 특히 Redis를 중간 저장소로 사용하기 때문에 문장별 낭독 분석과 퀴즈 생성 요청이 시간적으로 분리되어도 같은 학습 세션의 오류 데이터를 유지할 수 있다.

3.5 기타

3.5.1. OpenAI GPT-5.1 API 인터페이스

동화 텍스트 생성에는 OpenAI의 GPT-5.1 모델을 사용하였다. FastAPI의 LLMStoryGeneratorService가 AsyncOpenAI 클라이언트를 통해 비동기로 호출한다.

항목	내용
사용 모델	gpt-5.1 — 한국어 동화 생성 품질 및 JSON 출력 안정성 최적
API 클라이언트	openai Python 라이브러리의 AsyncOpenAI (비동기 호출)
API 메서드	client.chat.completions.create()
response_format	{"type": "json_object"} — GPT가 반드시 JSON만 출력하도록 강제
temperature	0.7 — 창의적이면서도 구조를 준수하는 균형점
응답 검증	Pydantic StoryResponse.model_validate_json()으로 스키마 검증
에러 처리	ValidationError(형식 불일치) → ValueError / 통신 오류 → RuntimeError

특히 response_format을 json_object로 지정하고, 추가로 Pydantic의 model_validate_json()으로 2중 검증하여 GPT의 JSON 출력 안정성을 극대화하였다. GPT가 지정된 스키마를 어길 경우 ValidationError가 발생하여 명확한 에러 메시지를 반환한다.

3.5.2 AI 프롬프팅 구현 현황

AI 프롬프팅은 단순히 자연어 지시문을 작성하는 작업이 아니라 아동의 레벨 제약과 오류 분석 결과를 LLM이 이해할 수 있는 명세로 변환하고, 다시 서버가 검증 가능한 JSON 구조로 회수하는 인터페이스 설계에 해당한다.

2-1) 프롬프트 설계 구조

구성 요소	역할	주요 내용
system prompt	모델의 역할과 절대 규칙 고정	난독 아동을 위한 한국어 동화 작가/언어 치료사 역할, JSON-only, 금지 어미, 문체 규칙, 출력 형식 제한
user prompt	세션별 동적 조건 전달	아동 레벨, 관심사 키워드, WCPM, 취약 음운, 페이지 수, 문장 길이, 퀴즈 오류 패턴 등
response_format	기계 파싱 가능성 확보	json_object 형식으로 응답을 제한하여 Pydantic 검증과 후속 모듈 연동이 가능하도록 함
post-validation	LLM 산출물 안정성 보장	StoryResponse/QuizResponse 스키마 검증, 중복 제거, 필수 key 확인, 예외 처리

LLM 응답을 그대로 사용자에게 보여주지 않고 서비스 내부 데이터 구조로 변환해 사용하는 방식이다. 따라서 프롬프트에는 교육적 문체뿐 아니라 서버가 요구하는 JSON 필드명과 배열 길이도 함께 명시된다.

2-2) 동화 생성 프롬프트

동화 생성 프롬프트는 build_story_prompt()에서 생성된다. system prompt는 모델의 역할과 출력 형식을 고정하고, user prompt는 ConstraintResponse의 값을 삽입하여 매 요청마다 다른 학습 조건을 반영한다.

- 역할 고정: “난독 아동을 위한 전문 한국어 동화 작가이자 특수교사”로 명시한다.
- 문체 제한: 모든 평서문 어미를 “-요”로 통일하고, “-다/합니다/해” 체를 금지하여 아동 친화적 문체를 유지한다.
- 문장 길이 제한: 한 페이지 1문장, 최대 max_sentence_len 어절, 필요 시 min_sentence_len 이상 조건을 삽입한다.
- 레벨별 분량 제한: total_pages, scene_count, sentences_per_scene을 명시하여 L1~L6 난이도별 분량을 제어한다.
- 취약 음운 반영: focus_phonemes를 자연스럽게 포함하되, 특정 음운이 과도하게 반복되어 어색한 문장이 되지 않도록 지시한다.
- 이미지 프롬프트 일관성: 모든 page image_prompt에 캐릭터 외형, 의상, 소지품, 화풍 정보를 영어로 반복 기재하도록 요구한다.
- 출력 스키마: title, cover_image_prompt, pages[].page_order, pages[].image_prompt, pages[].sentences[] 구조를 명시한다.

현재 구현에서는 L1~L6 모든 레벨에 대해 동일한 프롬프트 템플릿을 사용하되, `ConstraintResponse`에 들어가는 값만 바뀌도록 구현하였다. 이 방식은 프롬프트의 일관성을 유지하면서도 아동별 개인화 조건을 반영할 수 있다는 장점이 있다.

2-3) 음운 퀴즈 프롬프트

음운 퀴즈 프롬프트는 `generate_quiz_words(target_word, error_pattern)`에서 생성된다. `system prompt`에는 난독 아동 언어 치료사 역할을 부여하고, 토토리 내부 오류 표기법인 “X 탈락”, “X 첨가”, “X→Y 대치”의 의미를 함께 설명한다. 이를 통해 모델이 오류 패턴을 단순 문자열이 아니라 학습 목표로 해석하도록 하였다.

- 첫 단어는 아이가 실제로 틀린 `target_word`로 고정한다.
- 나머지 3개 단어는 같은 자음 또는 모음이 실제로 포함된 친숙한 단어로 생성한다.
- 4개 단어는 모두 달라야 하며, 중복 후보가 나오면 후처리에서 제거한다.
- 초등학생이 이해할 수 있는 실생활 어휘를 우선한다.
- 출력은 `{"words": [...]}` JSON 객체만 허용한다.

이 프롬프트는 저레벨 아동에게 발음 단위를 직접적으로 반복 노출시키는 데 초점을 둔다. 예를 들어 받침 “ㄱ” 탈락이 자주 발생하면 “책, 국, 약”처럼 동일 음운을 포함하는 단어를 생성하여 읽기 연습으로 연결한다.

2-4) 조사 퀴즈 프롬프트

조사 퀴즈 프롬프트는 `generate_josa_quiz(event)`에서 생성된다. 입력값인 `JosaEvent.kind`에 따라 오류 설명(`error_desc`)과 연습 조사(`practice_josa`)를 다르게 구성한다.

JosaEvent.kind	프롬프트 내 오류 설명	예시 학습 방향
DELETION	조사 X를 빠뜨리는 오류	체언 뒤에 필요한 조사를 붙여 읽는 연습
SUBSTITUTION	조사 X를 Y로 잘못 읽는 오류	의미에 맞는 올바른 조사를 구분하는 연습
INSERTION	없어야 할 조사 Y를 추가하는 오류	불필요한 조사를 붙이지 않고 자연스럽게 읽는 연습

조사 퀴즈는 문장 길이가 길어지면 난이도가 급격히 올라가기 때문에, 프롬프트에 두 어절 내외의 짧은 문장과 체언+조사 / 용언 구조를 명시하였다. 이를 통해 LLM이 복잡한 복문을 생성하는 비율을 줄이고, 조사 사용에 초점을 둔 문항을 생성하도록 유도하였다.

2-5) 관심사 키워드 정제는 비-LLM 방식으로 처리

관심사 키워드 정제는 GPT를 사용하지 않고 `KoNLPy Okt`와 사전 기반 필터링으로 처리한다. 이 결정은 비용 절감뿐 아니라 안전성 확보를 위한 설계이다. 아동의 자유 발화에는 부적절한 단어가 섞일 수 있으므로, LLM 호출 전에 불용어와 유해 키워드를 결정적으로 제거한 뒤 안전한 키워드만 동화 생성 프롬프트에 전달한다.

3.5.3. AI 엔진 간 데이터 흐름 요약

3개의 AI 엔진과 2개의 백엔드 서버 간의 데이터 흐름을 정리하면 다음과 같다.

- 1. iOS Swift → Spring Boot: 사용자 입력(sttText 또는 audioFile) 전달
- 2. Spring Boot → FastAPI: stt_text + level + recent_wcpm + weak_phonemes 전달
- 3. FastAPI 내부 파이프라인: InterestRefinerService(관심사 정제) → ConstraintGeneratorService(제약조건 생성) → prompt_builder(프롬프트 빌드) → LLMStoryGeneratorService(GPT-5.1 호출)
- 4. FastAPI → Spring Boot: StoryResponse JSON (title + cover_image_prompt + pages[image_prompt, sentences]) 반환
- 5. Spring Boot → Stability AI: 각 page의 image_prompt + 공통 seed로 이미지 병렬 생성 (@Async)
- 6. Spring Boot → ElevenLabs: 각 sentence의 text로 TTS 병렬 생성 (CompletableFuture)
- 7. Spring Boot → S3: 생성된 이미지·음성 파일을 S3에 업로드, Presigned URL 발급
- 8. Spring Boot → iOS Swift: BookGenerateResponse (bookId, title, coverImageUrl, pages[imageUrl, audioUrls]) 반환

3.5.4. 미완성 및 향후 개선 사항

현재 구현에서 완전히 완료되지 않은 부분과 향후 개선 계획은 다음과 같다.

- 레벨 자동 승급: 현재는 관리자/부모가 수동으로 레벨을 설정한다. 향후 WCPM 추이와 퀴즈 정답률을 분석하여 레벨을 자동 추천/승급하는 로직을 추가할 예정이다.
- 이미지 캐싱: 유사한 프롬프트에 대한 중복 API 호출이 발생할 수 있다. Redis 기반 프롬프트 해시 캐싱으로 API 비용 절감 및 응답 속도 개선을 계획하고 있다.
- 보상 방식 확대: 현재 ‘도토리’를 통한 보상 방식으로 아동의 참여를 유도하고 있지만 이와 같은 보상방식은 지속적인 참여를 유도하기에는 약하다고 생각한다. ‘도토리’를 통해 보상을 얻을 수 있는 등의 보상 방식 확대를 계획하고 있다.

참고문헌

[1] 마켓뉴스, “'읽기-학습장애'가 지능 부족은 아니다!”...국내 난독 아동 지원 시급”,
<http://www.marketnews.co.kr/news/articleView.html?idxno=72902>,
Accessed: 2025-11-06.

[2] 메타마인드, “코로나 시대 초등학생 난독증 폭증”,
<https://metamind.kr/news-events/코로나-시대-초등학생-난독증-폭증/>,
Accessed: 2025-12-14.